5

5. Компютърни архитектури, формати на данните и структура на централния процесор

1. Обща структура на компютрите и концептуално изпълнение

Обща структура на компютърната система

Персоналният компютър е изчислителна машина, предназначена за решаване на специфични алгоритмични задачи. Неговата архитектура се състои от няколко основни компонента:

- **Централен процесор (ЦП):** „Изчислителната сила“, която осъществява всички процеси.
- **Памет:**
  - *Вътрешна:* Необходима директно за процесора.
  - *Външна (HDD, SSD):* За дълготрайно съхранение на големи обеми информация.
  - *Оперативна (RAM):* Съхранява текущите програми и данни.
  - *Постоянна (ROM):* Съдържа служебна информация само за четене (напр. стартираща информация).
- **Периферни устройства:** Входно-изходни компоненти (мишка, клавиатура).
- **Мрежова карта (NIC):** За достъп до мрежата.

Организация на комуникацията: Информационният обмен се осъществява чрез линии, които могат да бъдат организирани като **индивидуални канали** (пълен граф) или **обща информационна шина**. Общата шина е по-гъвкава и евтина, но позволява комуникация само между два компонента едновременно. Тя се разделя на три подшини:

1. **За данни:** Прехвърля информация между ЦП и паметта/устройствата.
2. **За адреси:** Определя конкретното място в паметта (напр. 32-битова шина адресира до 4GB).
3. **Управляваща:** Пренася сигнали за контрол (четене, писане, синхронизация, прекъсвания).

Принцип на запомнената програма

Този основополагащ принцип гласи, че **инструкциите и данните се съхраняват в една и съща памет**. Това позволява на една и съща машина да изпълнява различни програми без промяна на хардуера, просто чрез зареждане на нова последователност от инструкции.

2. Формати на данните

Всички данни в паметта се представят в двоичен вид (битове). Смисълът на една последователност зависи от избрания формат:

Цели двоични числа

Код	Описание	Характеристики
Прав код	Най-старшият бит е знаков (0 за +, 1 за -). Останалите са абсолютна стойност.	Нулата има две представяния. Трудна апаратна реализация.
Обратен код	Отрицателните числа се образуват чрез инвертиране на битовете на абсолютната стойност.	Нулата има две представяния. При пренос от най-старшия бит се добавя 1 към резултата.
Допълнителен код	Инвертиране + добавяне на 1. Най-разпространеният метод.	Едно единствено представяне на нулата. Аритметичните операции са най-ефективни (преносът навън се игнорира).

Двоично-десетични числа (BCD)

Всяка десетична цифра (0-9) се кодира с 4 бита.

- **Непакетирана форма:** Една цифра в един байт.
- **Пакетирана форма:** Две цифри в един байт.
- *Предимство:* Лесно десетично преобразуване и финансова точност. *Недостатък:* Неикономичност (6 неизползвани комбинации в 4 бита).

Двоични числа с плаваща запетая

Използват се за реални числа чрез формулата: m \times 2^p

- **Мантиса (m):** Цяло число, съдържащо значещите битове.
- **Порядък (p):** Цяло число, определящо позицията на запетаята.
- **Стандарт (IEEE 754):** Обикновено 32 бита (8 за порядък, 24 за мантиса и знак). За по-голяма точност се използват 64-битови „двойни думи“.

Символни данни и кодови таблици

Всеки символ се асоциира с битова поредица:

- **ASCII:** Първоначално 7-битова, по-късно 8-битова (256 символа).
- **Unicode:** 16-битова (или променлива като UTF-8), поддържаща всички езици.
- **EBCDIC:** 8-битова таблица на IBM.

3. Вътрешна структура на централния процесор

Процесорът се състои от няколко критични блока:

Регистри

Свръхбърза памет за временни данни:

1. **С общо предназначение:** За междинни резултати и адресация.
2. **Служебни регистри:**
    - **PC (Program Counter):** Сочи към адреса на следващата инструкция.
    - **IR (Instruction Register):** Съхранява текущата инструкция.
    - **SP (Stack Pointer):** Сочи към върха на стека.
    - **MAR (Memory Address Register):** Съдържа адреса на клетката памет за следващата операция.
    - **MBR (Memory Buffer Register):** Буфер между ЦП и RAM.

Аритметико-логическо устройство (АЛУ)

Извършва пресмятанията. Основният му елемент е **двоичният суматор**. В модерните процесори има и отделен блок за работа с числа с плаваща запетая.

Регистър на състоянието и флагове

Съдържа индикатори (флагове) за резултата от последната операция:

- **Нулев флаг (Z):** Резултатът е 0.
- **Знаков флаг (S):** Резултатът е отрицателен.
- **Флаг за пренос (C):** Пренос при беззнакова аритметика.
- **Флаг за препълване (V/O):** Некоректен резултат при знакова аритметика.

4. Инструкции на централния процесор

Структура на инструкцията

- **Префикси:** Модифицират интерпретацията (напр. размер на операнда или повторение).
- **Код на операцията (Opcode):** Определя действието (събиране, преход и т.н.).
- **Местоположение на операндите:** Къде се намират данните (регистри, памет, самата инструкция).

Модели на адресация

- **Непосредствена:** Данните са в самата инструкция.
- **Регистрова:** Данните са в регистър.
- **Директна:** Инструкцията съдържа точния адрес в паметта.
- **Индиректна:** Инструкцията сочи към място, където е записан адресът на операнда.
- **Базова/Индексна:** Адресът се изчислява чрез събиране на базов регистър и отместване.

Типове инструкции

1. **Аритметико-логически:** +, -, *, логически И, ИЛИ, XOR, сравнения.

2. **Низови:** Обработка на последователности (копиране, сравнение).
3. **Безусловни и условни преходи:** Променят стойността на **PC**, за да реализират цикли и разклонения.
4. **Управление на програмата:** Извикване на подпрограми (**Call/Return**) и прекъсвания.
5. **Транспортни:** Прехвърляне на данни (памет-регистър, регистър-регистър).

Конвейерна обработка (Pipelining)

Техника за ускоряване, при която изпълнението на инструкциите се разделя на етапи (извличане, декодиране, изпълнение и т.н.). Докато една инструкция е на етап „изпълнение“, следващата се „декодира“.

7. Терминологичен речник (Глосарий)

- **ALU (Аритметико-логическо устройство):** Блок в ЦП за извършване на математически и логически операции.
- **ASCII:** 7 или 8-битов стандарт за кодиране на символи.
- **BCD (Binary-Coded Decimal):** Двоично-десетичен формат, кодиращ всяка цифра с 4 бита.
- **IEEE 754:** Международен стандарт за представяне на числа с плаваща запетая.
- **MAR (Memory Address Register):** Регистър, съхраняващ адреса на паметта, до която ще се осъществи достъп.
- **MBR (Memory Buffer Register):** Регистър-буфер, съдържащ данните, които се записват в или четат от паметта.
- **Opcode (Код на операцията):** Част от инструкцията, която казва на процесора какво действие да извърши.
- **Program Counter (PC):** Специален регистър, който винаги сочи към следващата команда в оперативната памет.
- **Unicode:** Универсален стандарт за кодиране, обхващащ символи от почти всички писмени системи.
- **Шина (Bus):** Споделена магистрала за обмен на данни, адреси и управляващи сигнали между компонентите.

6

6. Структура и йерархия на паметта. Сегментна и странична преадресация. Система за прекъсване

1. Йерархия на паметта

Йерархията на паметта е организирана в няколко нива, целящи да балансират три противоречиви изисквания: висок капацитет, голяма скорост и ниска цена.

Основни характеристики по нива

С изкачване в йерархията (към процесора):

- **Капацитет:** Намалява.
- **Скорост (време за достъп):** Нараства (времето за достъп намалява).
- **Цена за бит:** Нараства.

Разделение на паметта

1. **Вътрешна памет (Първична):** Директно достъпна за процесора. Включва регистрите на ЦП, кеш паметта и оперативната памет.
2. **Външна памет (Вторична):** Достъпна чрез входно-изходни модули. Включва HDD, SSD, оптични дискове и магнитни ленти. Участва в реализацията на виртуалната памет.

Компоненти на йерархията

- **Кеш памет:** Високоскоростна памет (SRAM), която съхранява най-често използваните данни/инструкции. Дели се на:
    - **L1:** Най-бърз, малък (8-64 KB), разделен на кеш за инструкции и кеш за данни.
    - **L2 и L3:** По-големи и по-бавни; L3 обикновено е споделен между ядрата.
    - **Cache Hit/Miss:** Ако данните са в кеша, имаме "hit" (попадение); ако не са – "miss" (пропуск). Ефективността се измерва с **Hit Ratio**.
- **Оперативна памет (RAM):** Съдържа текущо изпълняваните програми и данни. Използва DRAM (Dynamic RAM). Тя е волатилна (губи данни без ток).
- **Виртуална памет:** Механизъм, при който външната памет се адресира като част от основната. Позволява използването на по-голямо адресно пространство от физически наличното.

2. Сегментна преадресация

Сегментацията разглежда паметта като съвкупност от логически области с променлива дължина, наречени сегменти (напр. за код, данни, стек).

- **Компоненти на адреса:** Сегментен селектор и отместване (offset).
- **Сегментен селектор:** Индекс, който сочи към дескриптор в специална таблица.

- **Сегментен дескриптор:** Описва сегмента чрез неговия базов адрес (начало), граница (дължина/размер) и права за достъп.
- **Таблици и регистри:**
    - **Глобална дескрипторна таблица (ГДТ):** Единствена за системата, съдържа системни сегменти.
    - **Локална дескрипторна таблица (ЛДТ):** По една за всеки процес, съдържа приложни сегменти.
    - Процесорът използва специални **сегментни регистри**, в които се зареждат селекторите.

Процес на преадресация: Базовият адрес от дескриптора се събира с отместването, за да се получи линеен адрес. Ако отместването надвишава границата, се сигнализира грешка в сегментната защита.

3. Странична преадресация

Метод за управление (paging), при който виртуалното адресно пространство (ВАП) се дели на фрагменти с фиксиран размер – **страници**, а физическото (ФАП) – на **блокове (рамки)**.

- **Каталог (Таблица) на страниците:** Поддържа се от ОС и съдържа връзките между виртуалните и физическите адреси.
- **Описател на страница (Page Descriptor):** Един запис в таблицата. Съдържа номера на физическия блок и битове за състояние:
    - **Бит за наличност (валидност):** Показва дали страницата е в RAM.
    - **Dirty/Modified bit:** Показва дали страницата е била променяна.
- **Преобразуване:** Виртуалният адрес (номер на страница + отместване) се транслира чрез таблицата, като номерът на страницата става номер на блок, а отместването остава същото.

Стратегии за подмяна на страници

Когато RAM е пълна и трябва да се зареди нова страница:

1. **FIFO (First-In-First-Out):** Заменя се най-отдавна заредената страница.
2. **LRU (Least Recently Used):** Заменя се страницата, която не е ползвана най-дълго време.
3. **LFU (Least Frequently Used):** Заменя се най-рядко използваната страница.

4. Система за прекъсване

Прекъсването е механизъм за временно спиране на програмата, за да се обслужи събитие (външно или вътрешно).

Структура и обработка

1. **Източник:** Хардуер (мишка, таймер) или Софтуер (изключения, системни повиквания).
2. **Таблица на векторите на прекъсванията:** Област в паметта с указатели към **обработчици (рутини)**.
3. **Етапи на обслужване:**
    - Разпознаване на прекъсването от **контролера на прекъсванията**.
    - Запазване на текущото състояние (програмен брояч, регистри) в **стека**.
    - Изпълнение на обработчика.

    - Възстановяване на състоянието и възобновяване на програмата.

Типове прекъсвания

- **По машинна грешка:** Най-висок приоритет.
- **Входно-изходни:** Активират се при завършване на I/O операция.
- **Външни:** Свързани с външни елементи.
- **Програмно-активирани (SVC):** Специални команди за системни повиквания.
- **Изключения:** Делене на нула, препълване, липсваща страница.

Конкурентност и приоритети

- **Приоритети:** Определят реда на обслужване. По-ниските номера на прекъсвания обикновено имат по-висок приоритет. Немаскируемите прекъсвания имат предимство пред маскируемите.
- **Контролер на прекъсванията (напр. PIC):** Хардуер, който управлява заявките (IRQ), подрежда ги по важност и ги насочва към процесора.

Речник на ключовите термини

- **Бит (Bit):** Най-малката единица информация (0 или 1).
- **Байт (Byte):** 8 бита; основната адресируема единица памет.
- **Кеш памет (Cache Memory):** Малка, високоскоростна памет между ЦП и RAM.
- **Hit Ratio:** Съотношение между успешните попадения в кеша и общия брой заявки.
- **Сегментен селектор (Segment Selector):** Индекс, използван за избор на дескриптор от таблица.
- **Линеен адрес:** Адресът, получен след сегментната преадресация.
- **Таблица на страниците (Page Table):** Структура, съдържаща връзките между виртуални и физически адреси.
- **IRQ (Interrupt Request):** Заявка за машинно прекъсване от хардуерно устройство.
- **SVC (Supervisor Call):** Програмно-активирано прекъсване за системна услуга.
- **Локалност (Locality):** Свойство на програмите да ползват многократно едни и същи или близки адреси (времева и пространствена локалност).

7

7. Процеси и комуникация между тях в операционната система

I. Процеси и комуникационни канали: Основни абстракции

1. Дефиниция и същност на процеса

Процесът е една от фундаменталните абстракции в ОС. Неформално той се определя като **инстанция на програма в момент на изпълнение**. Докато програмата е пасивен обект (файл с инструкции), процесът е активен обект, който притежава:

- Текущо състояние и програмен брояч.
- Адресно пространство, данни, стек и регистри на процесора.
- Отворени файлове и служебна информация за управление.

Процесите могат да инициират подпроцеси (детински процеси), които са частична реплика на родителя и спорелят част от неговите ресурси. Основната цел на процеса като абстракция е да осигури дълготрайност на програмния код, изолирайки го от промените в хардуера.

2. Функционални изисквания към ОС

За да управлява ефективно процесите, ОС трябва да изпълнява следните изисквания:

- **Създаване и прекратяване:** Динамично управление на жизнения цикъл.
- **Изолация:** Предотвратяване на възможността един процес да повреди паметта на друг.
- **Планиране (Scheduling):** Разпределяне на процесорното време (time-sharing) чрез таймерни прекъсвания, създаващи илюзия за едновременна работа.
- **Комуникация и синхронизация:** Предоставяне на механизми за обмен на данни и координация.

3. Комуникационни канали

Комуникационният канал е средство за предаване на данни между процеси. ОС предоставя няколко основни вида:

Вид канал	Характеристики	Пространство на имената
Неименувана тръба (Pipe)	Създава се с `pipe(fd[2])`. Видима само за създателя и неговите наследници. Използва се само локално.	Не използва

Именувана тръба (FIFO)	Създава се с `mkfifo`. Видима за всички процеси в системата, независимо от роднинската им връзка.	Използва (има име)
Файл	Комуникация чрез `open`, `read` и `write`. Разглежда се като прост абстрактен процес за съхранение на данни.	Използва
Конекция (Socket)	Крайна точка за комуникация между отдалечени обекти (сървър и клиент). Може да свързва процеси в различни машини.	Използва (за сървъра)

II. Комуникация чрез споделена памет и Race Condition

1. Споделена памет

При този метод процесите споделят обща променлива или област от паметта. Това е изключително ефективен метод, тъй като информацията не преминава през ядрото на ОС. Коректността на комуникацията зависи изцяло от реализацията на програмиста.

2. Съревнование за ресурси (Race Condition)

Проблемът възниква, когато два или повече процеса имат достъп до споделени ресурси и се опитват да ги променят едновременно. Резултатът зависи от хаотичния ред на изпълнение, определен от алгоритъма за планиране.

- **Критична секция (Critical Region):** Частта от кода, в която се осъществява достъп и промяна на споделения ресурс.
- **Инвариант:** Временното му нарушаване по време на актуализация в критичната секция може да доведе до некоректни данни, ако друг процес влезе в секцията по същото време.

III. Хардуерна защита и Синхронизация чрез Spinlock

1. Хардуерна защита

В многопроцесорните системи простото забраняване на прекъсванията не е достатъчно. Използват се **атомарни инструкции** (като `test-and-set`), които гарантират, че проверката и промяната на даден бит (lock) се извършват като едно неделимо действие.

2. Spinlock (Активно изчакване)

Spinlock е метод, при който процесът "цикли" (busy waiting) и постоянно проверява стойността на заключващия бит, докато той стане 0.

- **Предимства:** Бърз механизъм за много кратки критични секции.

- **Недостатъци:** Хаби процесорно време без да върши полезна работа; неефективен при дълги секции; риск от безкрайни цикли при рекурсивни извиквания.

IV. Синхронизация от високо ниво: Семафори

Семафорът е абстрактен механизъм, въведен от Едгар Дейкстра, който решава проблемите на активното изчакване чрез приспиване на процесите.

1. Структура на семафора

- **Counter (Брояч):** Цяло число, показващо колко процеса могат да преминат. Ако е отрицателно, неговата абсолютна стойност показва броя на приспаните процеси.
- **List (Списък/Опашка):** Контейнер с приспаните процеси.

2. Основни методи

1. **init(cnt):** Инициализира брояча.
2. **wait() (P-операция):** Декрементира брояча. Ако стойността стане \le 0, процесът се блокира и влиза в списъка.
3. **signal() (V-операция):** Инкрементира брояча. Ако има чакащи процеси, един от тях се събужда.

V. Приспиване и събуждане (Block/Wakeup)

Процесите могат да преминат в състояние **sleeping/blocked** поради няколко причини:

- Очакване на I/O операция или събитие във времето (таймер).
- Очакване на сигнал от друг процес.
- Липсваща страница в паметта (page fault).
- Зает семафор.

При събуждане ядрото премества процеса от списъка на спящите в списъка на готовите за изпълнение (**ready to run**). Важно е да се отбележи, че събуждането не означава незабавно изпълнение, а само допустимост за планиране.

VI. Реализация на комуникационна тръба с n елемента

За реализацията на тръба (pipe), съхраняваща n байта, се използват три семафора, които координират работата на изпращачи (producers) и получатели (consumers):

Използвани инструменти:

1. **Опашка Q** с капацитет n.
2. **Семафор `free_bytes`:** Инициализиран с n (брои свободните места).
3. **Семафор `ready_bytes`:** Инициализиран с 0 (брои готовите за четене байтове).
4. **Мутекси `mutex_read` и `mutex_write`:** Инициализирани с 1 (осигуряват взаимно изключване).

Алгоритъм на работа:

Операция	Логика на изпълнение
Писане (Write)	`free_bytes.wait()` \rightarrow `mutex_write.wait()` \rightarrow **Добавяне на байт в Q** \rightarrow `mutex_write.signal()` \rightarrow `ready_bytes.signal()`
Четене (Read)	`ready_bytes.wait()` \rightarrow `mutex_read.wait()` \rightarrow **Вземане на байт от Q** \rightarrow `mutex_read.signal()` \rightarrow `free_bytes.signal()`

Важно: Първо се чака условието (свободно място/наличен байт) и едва след това се заема мутексът, за да се избегне deadlock.

Глосарий на ключовите термини

- **Атомарна операция:** Операция, която се изпълнява неделимо и не може да бъде прекъсната; никое междинно състояние не е видимо за други процеси.
- **Взаимно изключване (Mutual Exclusion):** Правило, според което най-много един процес може да бъде в своята критична секция за даден ресурс в даден момент.
- **Критична секция:** Част от програмния код, в която се осъществява достъп до споделен ресурс.
- **Мутекс (Mutex):** Бинарен семафор или механизъм, използван изключително за взаимно изключване.
- **Планировчик (Task Scheduler):** Структура в ОС, която решава кога да превключи даден процес от чакащо в работещо състояние.
- **Процес:** Програма в ход на изпълнение; активна машина за обработка на информация.
- **Семафор:** Абстрактен тип данни за синхронизация, включващ брояч и списък с блокирани процеси.
- **Съревнование за ресурси (Race Condition):** Некоректно състояние, при което крайният резултат зависи от непредсказуемия ред на изпълнение на няколко процеса.
- **Spinlock:** Механизъм за синхронизация от ниско ниво, базиран на активно изчакване в цикъл.
- **Deadlock (Мъртва хватка):** Ситуация, в която два или повече процеса се блокират взаимно, чакайки ресурси, държани от другия.

8

8. Компютърни мрежи и протоколи: OSI модел и основни интернет протоколи

1. OSI модел – най-обща характеристика на нивата

OSI (Open Systems Interconnection) е теоретичен модел, описващ принципите на комуникация в мрежа чрез седем независими слоя. Всеки слой предоставя услуга на слоя над него и използва интерфейс за взаимодействие.

Характеристика на седемте слоя:

№	Слой	Кратка характеристика и основни функции
7	**Приложен (Application)**	Най-близкият слой до потребителя. Позволява на приложенията да заявяват услуги. Протоколи: HTTP, DNS, FTP, SMTP.
6	**Презентационен (Presentation)**	Грижи се за формата на данните: транслация (конвертиране), криптиране/декриптиране и компресия.
5	**Сесиен (Session)**	Управлява диалога между приложенията – създаване, поддържане, синхронизация (контролни точки) и терминиране на сесии.
4	**Транспортен (Transport)**	Осигурява надеждно предаване на данни между крайни точки. Извършва сегментация, контрол на потока и поправка на грешки. (TCP, UDP).
3	**Мрежови (Network)**	Отговаря за маршрутизацията и логическата адресация. Намира най-краткия път между хостове в различни мрежи. (IPv4, IPv6).
2	**Канален (Data Link)**	Грижи се за безгрешно предаване на кадри (frames) между устройства в една мрежа чрез MAC адреси. Подслоеве: LLC и MAC.

1	**Физически (Physical)**	Предава неструктуриран поток от битове по физическия носител (кабели, радиовълни). Дефинира топология и режим на предаване.

2. Съпоставяне на OSI модела с модела TCP/IP

Докато OSI е концептуална рамка, **TCP/IP** е практическият стандарт, върху който е изграден Интернет.

- **Компактност:** TCP/IP има 4 слоя вместо 7.
- **Съответствие:**
  - **Слой Приложение (TCP/IP)** обединява Приложен, Презентационен и Сесиен от OSI.
  - **Слой Транспорт (TCP/IP)** съответства директно на Транспортния слой от OSI.
  - **Слой Маршрутизация/Интернет (TCP/IP)** съответства на Мрежовия слой от OSI.
  - **Слой Достъп до мрежата (TCP/IP)** обединява Каналния и Физическия слой от OSI.

Ключови разлики: OSI прави строго разграничение между услуги, интерфейси и протоколи, докато TCP/IP е изграден на базата на вече съществуващи протоколи и е по-труден за подмяна на компоненти.

3. IPv4 адресация – класова и безкласова

IPv4 адресите са 32-битови числа, представени като четири октета (напр. 192.168.1.1).

Класова адресация

Първоначално адресното пространство е разделено на пет основни класа:

- **Клас A:** (1.0.0.0 – 127.255.255.255). За огромни мрежи. Първият октет е за мрежа (префикс 0).
- **Клас B:** (128.0.0.0 – 191.255.255.255). За средни мрежи. Първите два октета са за мрежа (префикс 10).
- **Клас C:** (192.0.0.0 – 223.255.255.255). За малки мрежи. Първите три октета са за мрежа (префикс 110).
- **Клас D:** (224.0.0.0 – 239.255.255.255). Резервиран за мултикастинг (предаване към група).
- **Клас E:** (240.0.0.0 – 255.255.255.254). Резервиран за бъдещи/експериментални цели.

Безкласова адресация (CIDR)

Поради неефективното разпределение при класовете е въведен **CIDR (Classless Inter-Domain Routing)**. Той използва маска с произволна дължина, отбелязана след наклонена черта (напр. `/24`). Това позволява гъвкавост и по-малки маршрутни таблици чрез агрегиране.

Резервирани IPv4 адреси:

- `0.0.0.0`: Маршрут по подразбиране или неизвестен адрес.
- `127.0.0.1`: Loopback адрес за локалния хост (localhost).

- **Частни мрежи:** `10.x.x.x`, `172.16.x.x - 172.31.x.x`, `192.168.x.x` (не се рутират в Интернет).

4. Основни характеристики на протокол IPv6

IPv6 е създаден, за да реши проблема с изчерпването на IPv4 адресите. Той използва **128-битови адреси**, записвани в шестнадесетичен формат (8 групи по 4 цифри).

Основни предимства:

1. **Огромно адресно пространство:** Около 3.4 \times 10^{38} адреса.
2. **Опростен хедър:** Фиксиран размер и по-малко полета, което ускорява рутирането.
3. **Комуникация от край-до-край:** Елиминира нуждата от NAT, тъй като всяко устройство може да има уникален глобален адрес.
4. **Автоматична конфигурация:** Stateless (SLAAC) и stateful конфигурация без нужда от външни услуги.
5. **Сигурност:** Вградена поддръжка на IPsec.
6. **Липса на Broadcast:** Използва се Multicast за по-голяма ефективност.
7. **Anycast:** Възможност за насочване към най-близкия от група хостове с един и същ адрес.

5. TCP – процедура на трикратно договаряне

TCP (Transmission Control Protocol) е надежден, ориентиран към връзка протокол на транспортното ниво. Преди обмен на данни се извършва **трикратно договаряне (three-way handshake)**:

1. **SYN (Synchronize):** Клиентът изпраща сегмент с вдигнат SYN флаг и начален пореден номер (Sequence Number), за да заяви отваряне на връзка.
2. **SYN + ACK (Synchronize-Acknowledgment):** Сървърът отговаря със сегмент, който потвърждава клиентския номер (ACK) и изпраща свой собствен начален пореден номер (SYN).
3. **ACK (Acknowledgment):** Клиентът потвърждава получения от сървъра пореден номер.

След тези стъпки се установява пълнодуплексна връзка и започва надеждният пренос на данни.

6. Хипертекстов протокол HTTP

HTTP (Hypertext Transfer Protocol) е текстов протокол на приложния слой, работещ на модела клиент-сървър (обикновено на порт 80).

Структура на заявката:

- **Метод на достъп:** Определя действието.
- **Request URI:** Идентифицира ресурса.
- **Header полета:** Метаданни за заявката.

Основни HTTP методи:

- **GET:** Извличане на данни (репрезентация на ресурс).
- **POST:** Изпращане на данни към сървъра (създаване/промяна).

- **PUT:** Замяна на ресурс.
- **PATCH:** Частична модификация.
- **DELETE:** Изтриване на ресурс.
- **HEAD:** Подобен на GET, но връща само заглавната част (без тяло).

Статус кодове (отговор):

- **1xx:** Информационни.
- **2xx:** Успех (напр. 200 OK).
- **3xx:** Пренасочване.
- **4xx:** Клиентска грешка (напр. 404 Not Found).
- **5xx:** Сървърна грешка (напр. 500 Internal Server Error).

Речник на ключовите термини

- **MAC адрес:** Физически адрес на мрежовата карта, използван на каналното ниво.
- **IP адрес:** Логически адрес за идентификация на устройство в мрежата.
- **CIDR:** Метод за безкласова адресация, позволяващ гъвкаво разделяне на IP мрежи.
- **NAT (Network Address Translation):** Технология за превод на частни адреси в публични (ненужна при IPv6).
- **Трикратно договаряне:** Процес за синхронизация и установяване на TCP връзка.
- **Full-duplex:** Режим на комуникация, при който данни могат да се изпращат и получават едновременно.
- **URI (Uniform Resource Identifier):** Уникален идентификатор на ресурс в мрежата.
- **SLAAC:** Механизъм в IPv6 за автоматична конфигурация на адреси без сървър.

9

9. Бази от данни и релационен модел на данните

1. Релационен модел на данните: Основни понятия

Релационният модел, предложен от Едгар Код през 1969 г., е най-разпространеният начин за логическо организиране на данни. Той представя информацията под формата на двуизмерни таблици.

Компоненти на релационния модел

- **Релация (Relation):** Двуизмерна таблица, съставена от редове и колони. Тя е единственият начин за представяне на данни в този модел.
- **Атрибути (Attributes):** Имената на колоните в релацията. Те описват свойствата или характеристиките на обектите (например: `Title`, `Year`, `Length`).
- **Кортежи (Tuples):** Редовете в таблицата (без заглавния ред). Всеки кортеж представлява конкретен запис или обект и съдържа стойности за всеки от атрибутите.
- **Домейн (Domain):** Множеството от всички допустими стойности за даден атрибут. Релационният модел изисква стойностите в домейните да бъдат **атомарни** (елементарни, неразбиваеми). Не се допускат масиви, структури или списъци като стойност в една клетка.
- **Ключ на релацията (Key):** Множество от един или повече атрибути, които еднозначно (уникално) идентифицират всеки кортеж в релацията.

Схеми

- **Схема на релация:** Името на релацията, последвано от списъка с нейните атрибути, заградени в скоби.
  - *Пример:* `Movies(title, year, length, genre)`.
- **Схема на релационна база от данни:** Множеството от всички схеми на релации в рамките на една база данни и връзките между тях.

2. Реализация на релационна база от данни

Процесът на проектиране и внедряване преминава през няколко ключови етапа:

1. **Определяне на данните:** Идентифициране на обектите и техните характеристики.
2. **Определяне на връзките:** Дефиниране как обектите си взаимодействат.
3. **Определяне на ключове:** Избор на атрибути за уникална идентификация.
4. **Определяне на ограничения:** Налагане на правила върху обектите и връзките.
5. **Нормализация:** Отстраняване на излишества (редундантност) и аномалии.
6. **Техническа реализация:** Създаване на самата база данни чрез СУБД (Система за управление на бази данни).

Основни правила при създаване:

- Всеки клас обекти се представя като релация.

- Връзките между класовете се представят чрез релации, включващи ключовите атрибути на свързаните класове.

3. Операции и заявки към релационната база данни

Заявките са изрази, чрез които се манипулират или извличат данни. Те се реализират чрез езика **SQL (Structured Query Language)**, който се дели на няколко под-езика:

DML (Data Manipulation Language) – Манипулиране на данни

Тези операции работят върху кортежите и не променят структурата (схемата).

Операция	Описание	Пример
SELECT	Извличане на данни	`SELECT Title FROM Movies;`
INSERT	Добавяне на нов кортеж	`INSERT INTO Movies VALUES (...);`
UPDATE	Промяна на съществуващи данни	`UPDATE Movies SET Length = 120;`
DELETE	Изтриване на кортежи	`DELETE FROM Movies WHERE Year < 2000;`

*Важно: За окончателно запазване на промените при DML често е необходима командата **COMMIT**.*

DDL (Data Definition Language) – Дефиниране на данни

Тези операции променят схемата на релацията или базата данни. Промените се прилагат незабавно.

- **CREATE TABLE:** Създаване на нова релация и дефиниране на нейните атрибути и типове данни.
- **ALTER TABLE:** Промяна на структурата (добавяне или премахване на колони/ограничения).
- **DROP TABLE:** Пълно изтриване на релация от базата данни.

4. Релационна алгебра

Релационната алгебра е теоретичната основа на SQL. Тя се състои от оператори, които приемат една или две релации като операнди и връщат нова релация като резултат.

Основни операции

1. **Обединение (R ∪ S):** Връща кортежи, които са в R, в S или и в двете. Релациите трябва да имат еднакви схеми.
2. **Разлика (R \setminus S):** Връща кортежи, които са в R, но не и в S.
3. **Декартово произведение (R × S):** Комбинира всеки кортеж от R с всеки кортеж от S. Резултатът има голям брой кортежи и атрибути.
4. **Проекция (π_L(R)):** Премахва определени колони и оставя само тези, изброени в списъка L.
5. **Селекция (σ_C(R)):** Избира само тези редове (кортежи), които отговарят на условието C.

Допълнителни операции

- **Сечение (R ∩ S):** Връща само общите кортежи за двете релации.
- **Съединение (Join - ⋈):** Селективно сдвояване на кортежи от две релации въз основа на условие.
    - **Вътрешно съединение (Inner Join):** Връща само съвпадащите записи.
    - **Естествено съединение (Natural Join):** Съединява релациите по атрибути с еднакви имена и автоматично премахва повтарящите се колони.
- **Външни съединения (Outer Joins):**
    - **Ляво (Left):** Запазва всички кортежи от лявата релация, като запълва с `NULL` липсващите данни от дясната.
    - **Дясно (Right):** Дуално на лявото – запазва всичко от дясната релация.
    - **Пълно (Full):** Запазва всички записи от двете релации, запълвайки с `NULL` там, където няма съответствие.

7. Глосар на ключовите термини

- **Агрегатни оператори:** Функции като `min()`, `max()`, `avg()`, които връщат една стойност, изчислена върху множество от кортежи.
- **Аномалия:** Проблем при актуализация, добавяне или изтриване на данни, произтичащ от лошо проектирана схема (излишество).
- **Домейн:** Множество от всички допустими стойности за конкретен атрибут (например домейнът на `Year` са целите числа).
- **Кортеж (Tuple):** Ред в релация; представяне на конкретен обект или запис.
- **Ключ:** Минимално множество от атрибути, чиито стойности уникално идентифицират всеки ред.
- **Нормализация:** Процес на организиране на данните за намаляване на повторенията и подобряване на целостта.
- **Релация:** Двуизмерна структура (таблица), състояща се от атрибути (колони) и кортежи (редове).
- **СУБД (DBMS):** Софтуерна система за управление на бази данни (напр. MySQL, Oracle, SQL Server).
- **NULL:** Специална стойност, указваща липса на данни или неизвестна стойност.

10

10. Бази от данни и нормални форми

I. Проектиране на схеми, аномалии и ограничения

1. Проектиране на схеми на релационни бази от данни

Проектирането е процес на транслация на даннов модел (например същност-връзка) в съвкупност от релации. Основните цели са:

- **Запазване на цялостта на данните.**
- **Нормализация:** Реструктуриране на релациите за минимизиране на дублирането на данни.
- **Методи:** Синтез (създаване на нов дизайн) и Декомпозиция (разделяне на съществуваща релация на по-малки).

2. Аномалии при лошо проектиране

Лошо проектираните схеми водят до четири основни проблема:

- **Излишество (Redundancy):** Повторение на информация без нужда.
- **Аномалия при обновяване:** При промяна на данни в един кортеж, свързаните с него кортежи не се обновяват автоматично, което води до неконсистенция.
- **Аномалия при добавяне:** Невъзможност да се добави информация за един обект, без да се добави несвързана информация за друг (често при транзитивни зависимости).
- **Аномалия при изтриване:** Неволна загуба на важна информация при изтриване на конкретен запис.

3. Ограничения (Constraints)

Механизми за налагане на правила върху данните:

- **Уникалност:** Два кортежа не могат да имат еднакви стойности за ключови атрибути (`UNIQUE`, `PRIMARY KEY`).
- **Домейнни ограничения:** Стойностите трябва да са от определен тип или диапазон (`CHECK`, `NOT NULL`).
- **Външен ключ (`FOREIGN KEY`):** Осигурява връзка между релациите, като изисква стойността в едната релация да съществува като ключ в другата.

II. Функционални зависимости и ключове

1. Функционални зависимости (ФЗ)

Дефиниция: Множество атрибути X функционално определя Y (X \to Y), ако за всеки два кортежа, които съвпадат по стойностите на X, те задължително съвпадат и по стойностите на Y.

Основни правила за ФЗ: | Правило | Описание | | :--- | :--- | | **Разделяне** | A, B \to C, D може да се раздели на A, B \to C и A, B \to D. | | **Комбиниране** | A, B \to C и A, B \to D могат да се обединят в A,

B \to C, D. | | **Тривиална ФЗ** | Зависимост X \to Y, при която Y \subseteq X (десните атрибути са подмножество на левите). |

2. Аксиоми на Армстронг

Това са правила за генериране на нови ФЗ:

1. **Рефлексивност:** Ако Y \subseteq X, то X \to Y.
2. **Умножение (Разширение):** Ако X \to Y, то XZ \to YZ.
3. **Транзитивност:** Ако X \to Y и Y \to Z, то X \to Z.

3. Ключове и суперключове

- **Ключ:** Минимално множество от атрибути, което функционално определя всички останали атрибути в релацията.
- **Суперключ:** Множество от атрибути, което съдържа ключ в себе си. Той определя всички атрибути, но не е задължително минимален.
- **Кандидат-ключ:** Всеки минимален ключ (може да има няколко).
- **Първичен ключ:** Избраният от проектанта кандидат-ключ.

III. Нормални форми (от 1НФ до НФБК)

Процесът на нормализация следва стъпки за премахване на специфични зависимости:

Нормална форма	Изисквания	Премахва
1НФ	Всички стойности (атрибути) трябва да са атомарни.	Повтарящи се групи и списъци в клетките.
2НФ	В 1НФ + всеки непървичен атрибут зависи напълно от целия ключ.	Частични зависимости от ключа.
3НФ	В 2НФ + няма транзитивни зависимости на непървични атрибути.	Транзитивни ФЗ (непървичен атрибут определя друг непървичен).
НФБК (BCNF)	За всяка нетривиална ФЗ X \to Y, X трябва да е суперключ.	Аномалии, произтичащи от ФЗ, които не са от ключове.

IV. Многозначни зависимости и Четвърта нормална форма

1. Многозначни зависимости (МФЗ)

Означават се с X \twoheadrightarrow Y. Възникват, когато за една стойност на X има множество независими стойности на Y, които не зависят от останалите атрибути Z.

- **Пример:** Лице, говорещо няколко езика и имащо няколко хобита (езиците и хобитата са независими едно от друго).
- **Тривиална МФЗ:** Когато Y \subseteq X или X \cup Y обхваща всички атрибути на релацията.

2. Съединение без загуба (Lossless Join)

Декомпозицията на R на R_1 и R_2 е със съединение без загуба, ако R_1 \cap R_2 функционално определя или R_1, или R_2. Това гарантира, че при операцията `Natural Join` ще се възстанови точно оригиналната релация без "фалшиви" кортежи.

3. Четвърта нормална форма (4НФ)

Релацията е в 4НФ, ако е в НФБК и за всяка нетривиална МФЗ X \twoheadrightarrow Y, X е суперключ. Тя премахва излишествата, причинени от независими многозначни факти в една таблица.

VIII. Глосарий на ключовите термини

- **Атомарност:** Свойство на стойностите в базата данни да бъдат неделими (базово изискване за 1НФ).
- **Декомпозиция:** Процес на разделяне на една релация на няколко по-малки с цел нормализация.
- **Домейн:** Множество от допустими стойности за даден атрибут.
- **Кортеж (Tuple):** Един ред в релацията (таблицата).
- **Многозначна зависимост (МФЗ):** Зависимост, при която един атрибут определя множество от независими стойности за друг атрибут.
- **Непървичен атрибут:** Атрибут, който не участва в нито един кандидат-ключ.
- **Нормализация:** Реструктуриране на данните за избягване на дублиране и аномалии.
- **Покритие на атрибути (Closure):** Множеството от всички атрибути, които могат да бъдат функционално изведени от дадено подмножество.
- **Проекция:** Операция, при която се избират само определени колони (атрибути) от релацията.
- **Релация:** Двумерна таблица, съставена от атрибути (колони) и кортежи (редове).
- **Схема на релация:** Името на релацията и списъкът с нейните атрибути.
- **Транзитивна зависимост:** Когато непървичен атрибут зависи от ключа чрез друг непървичен атрибут.
- **Частична зависимост:** Зависимост на непървичен атрибут само от част от съставен ключ (нарушава 2НФ).

11

11. Процедурно програмиране – основни конструкции

1. Принципи на структурното програмиране

Процедурното програмиране е парадигма, която третира програмата като поредица от инструкции (стъпка по стъпка), организирани в процедури или функции. Структурното програмиране е негов частен случай, който се базира на следните основни принципи:

- **Модулност:** Разбиване на сложната задача на по-малки, управляеми части (подпрограми/функции), които са лесни за тестване и модифициране.
- **Йерархичен подход:** Използва се линеен подход „отгоре-надолу“ (top-down), при който данните и процедурите се разглеждат като различни единици.
- **Основни конструкции:** Програмите се изграждат само чрез три типа структури:
    1. **Последователност:** Изпълнение на инструкциите една след друга.
    2. **Избор (Разклонение):** Изпълнение на код според булево условие.
    3. **Повторение (Цикъл):** Многократно изпълнение на група инструкции.
- **Избягване на `goto`:** Ключова разлика на структурното програмиране е отказът от оператора за произволен скок (`goto`), което прави потока на програмата по-лесен за проследяване и анализ.

2. Управление на изчислителния процес

Управлението на процеса определя реда на изпълнение на инструкциите. Основните конструкции са условните оператори и операторите за цикъл.

Условни оператори (Разклонения)

- **`if` / `if-else`:** Изпълнява оператор, ако дадено булево условие е истина (`true`). `if-else` позволява избор между две алтернативи.
- **`switch`:** Оператор за множествен избор, който работи с целочислени типове. Използва `case` етикети и ключова дума `break` за прекратяване на изпълнението в избрания клон. Ако няма съвпадение, се изпълнява секцията `default`.
- **Тернарен оператор (`?:`):** Компактна форма на `if-else`. Синтаксис: `<условие> ? <израз1> : <израз2>`.

Оператори за цикъл (Повторения)

Цикличните процеси биват два вида: **индуктивни** (с предварително известен брой повторения) и **итеративни** (броят повторения зависи от условие).

Оператор	Тип условие	Характеристика

`while`	Пред-условие	Проверява условието преди всяко изпълнение. Може да не се изпълни нито веднъж.
`do-while`	Пост-условие	Проверява условието след изпълнение на тялото. Гарантира поне едно изпълнение.
`for`	Комбиниран	Включва инициализация, условие и корекция. Най-често се ползва за индуктивни процеси.

Специални оператори в цикли:

- `break`: Принудително излизане от най-вътрешния цикъл.
- `continue`: Прескачане на текущата итерация и преминаване към следващата.

3. Променливи – видове и присвояване

Променливите са именувани области в паметта със специфичен тип, име и стойност.

Видове според областта на видимост (Scope):

1. **Локални променливи:** Дефинирани вътре в блок или функция. Видими са само в този блок. Жизненият им цикъл приключва с края на блока.
2. **Глобални променливи:** Дефинирани извън всички функции. Достъпни са от всяка част на програмата до края на нейното изпълнение.

Инициализация и присвояване:

- **Инициализация:** Задаване на начална стойност при дефиниране. Неинициализираните променливи имат непредвидима стойност.
- **Оператор за присвояване (`=`):** Прехвърля стойността на десния операнд (`rvalue`) към левия (`lvalue`). Операторът е **дясноасоциативен** (напр. `a = b = 5`).
- **`lvalue` vs `rvalue`:** `lvalue` е място в паметта, чиято стойност може да се променя (променлива), докато `rvalue` е временна стойност (константа, литерал, резултат от изчисление).
- **Съкратени оператори:** `+=`, `-=`, `*=`, `/=`.
- **Инкрементиране/Декрементиране:**
  - `++a` (префикс): Първо увеличава, после връща стойността (`lvalue`).
  - `a++` (постфикс): Първо връща старата стойност, после увеличава (`rvalue`).

4. Функции и процедури

Функциите са относително независими части от програмата, които изпълняват определена задача и могат да се преизползват.

Основни понятия:

- **Сигнатура:** Включва името на функцията, типа на връщания резултат и списъка с параметри.
- **Процедура:** Подпрограма, която изпълнява действия, но не връща стойност (в C++ се дефинира като `void` функция).
- **Формални параметри:** Променливите, описани в дефиницията на функцията.
- **Фактически параметри (аргументи):** Реалните стойности, подадени при извикването.

Предаване на параметри:

1. **По стойност (Call by value):** Създава се локално копие на аргумента. Промените във функцията не влияят на оригинала.
2. **По референция/псевдоним (Call by reference):** Използва се оператор `&` в дефиницията. Функция работи директно с оригиналната променлива. Фактическият параметър трябва да е `lvalue`.
3. **Чрез указател (Call by pointer):** Предава се адресът на променливата. Промените през дереференциран указател влияят на оригинала.
4. **По име (Call by name):** Аргументът се замества в тялото и се оценява в контекста на извикването (характерно за по-стари езици или специфични логически анализи).

Рекурсия:

Функция, която извиква сама себе си. Изисква **дъно** (условие за край), за да не препълни стека.

- **Пряка:** `f()` извиква `f()`.
- **Косвена:** `f()` извиква `g()`, която извиква `f()`.
- **Линейна:** Едно рекурсивно извикване (напр. факториел).
- **Разклонена:** Повече от едно извикване (напр. Фибоначи).

Речник на основните термини

Термин	Дефиниция
Идентификатор	Поредица от символи (букви, цифри, долна черта), служеща за име на променлива или функция.
nullptr	Специална неутрална стойност за указател, указваща, че той не сочи никъде.
Сигнатура	Уникалната характеристика на функция, включваща името ѝ и типа/реда на нейните параметри.

Стекова рамка	Сегмент от паметта (в стека), създаван при извикване на функция за нейните локални променливи и параметри.
Дереферииране	Операция (`*ptr`), чрез която се достъпва стойността на адреса, съхраняван в указателя.
ASCII	Стандарт за кодиране на символи, при който всеки символ се представя с числово съответствие (обикновено 1 байт).
lvalue	Израз, който реферира към конкретно място в паметта и може да стои отляво на оператора за присвояване.
rvalue	Временна стойност, която не заема постоянно място в паметта и обикновено стои отдясно на оператора за присвояване.
Терминираща нула (`\0`)	Символ с ASCII код 0, използван в C++ за отбелязване края на символен низ в масив от тип `char`.

12

12. Процедурно програмиране – указатели, масиви и рекурсия

1. Указатели и указателна аритметика

Същност на указателите

Указателят е променлива, чиято стойност се интерпретира като адрес в паметта на друг обект. Докато обикновената променлива съдържа директно стойност (число, символ), указателят съдържа информация за мястото, където се намира тази стойност.

- **Тип на указателя:** Определя се от типа на данните, които адресира (например `int*`, `char*`, `double*`). Типът е критичен, защото указва как да се интерпретират байтовете на съответния адрес и каква е тяхната големина.
- **Размер в паметта:** Указателите заемат константна памет – 4 байта при 32-битова архитектура и 8 байта при 64-битова архитектура.
- **Неутрална стойност:** Използва се ключовата дума `nullptr` (или `NULL`), за да се означи, че указателят не сочи към валиден адрес.

Основни операции

1. **Реферирane (`&`):** Оператор за вземане на адрес. Ако `x` е променлива, `&x` връща нейния адрес.
2. **Дереференциране (`*`):** Оператор за достъп до стойността на адреса. Ако `p` е указател, чрез `*p` можем да четем или променяме стойността в паметта, към която той сочи.
3. **Указателна аритметика:** Позволява движение през последователни обекти в паметта.
    - `p + n`: Връща адрес, отместен с `n * sizeof(T)` байта напред.
    - `p++` / `++p`: Премества указателя към следващия елемент от съответния тип.
4. **Сравнение:** Указателите могат да се сравняват чрез логически оператори (`==`, `!=`, `<`, `>`, `<=`, `>=`).

Специални видове указатели и референции

- **Указател към неизвестен тип (`void*`):** Може да сочи към променлива от произволен тип, но не може да се дереференцира директно и не поддържа указателна аритметика.
- **Константни указатели:**
    - `T* const`: Константен указател – самият адрес не може да се променя, но стойността на адреса може.
    - `const T*`: Указател към константа – адресът може да се променя, но стойността, към която сочи, е защитена от промяна.
- **Псевдоними (References - `T&`):** Алтернативно име на съществуващ обект. За разлика от указателите, псевдонимът трябва да бъде инициализиран при дефинирането си и не може да бъде пренасочван.

2. Едномерни и многомерни масиви

Едномерни масиви

Масивът е последователност от елементи от един и същ тип, разположени последователно в паметта.

- **Индексиране:** Достъпът се осъществява чрез индекс, започващ от `0` до `size - 1`. Изразът `arr[i]` е еквивалентен на `*(arr + i)`.
- **Характеристики:** Името на масива се преобразува в указател към първия му елемент. Масивите не могат да се сравняват или копират директно (например `a = b` е невалидно); тези операции се извършват поелементно.

Многомерни масиви (Матрици)

Двумерният масив е масив, чиито елементи са масиви.

- **Логическо представяне:** Таблица с редове и стълбове.
- **Физическо представяне:** В паметта елементите се съхраняват линейно, ред по ред.
- **Индексиране:** `matrix[i][j]` осъществява достъп до елемента на ред `i` и колона `j`.

Динамични масиви

Когато размерът не е известен по време на компилация, се използва динамична памет (heap):

- **Заделяне:** `int* ptr = new int[N];`
- **Освобождаване:** `delete[] ptr;` (задължително, за да се избегне изтичане на памет - *memory leak*).

Основни алгоритми в едномерен масив

Алгоритъм	Описание	Сложност
Линейно търсене	Проверка на елементите един по един от началото до края.	O(n)
Двоично търсене	Изисква сортиран масив. Разделя интервала на две при всяка стъпка.	O(log n)
Метод на мехурчето	Сравнява съседни елементи и ги разменя, докато най-големият "изплува" в края.	O(n²)
Пряка селекция	Намира най-малкия елемент в несортираната част и го поставя на текуща позиция.	O(n²)

Сортиране чрез вмъкване	Вмъква всеки следващ елемент на правилното място в сортираната до момента част.	O(n²)
Бързо сортиране (Quicksort)	"Разделяй и владей" – избира се опора (pivot) и се сортират подмасивите около нея.	O(n log n) средно

3. Символни низове

Представяне в паметта

Символните низове в C-стил са масиви от типа `char`, които завършват със специален терминиращ символ `'\0'` (нулев терминатор).

- **Размер:** За съхраняването на низ с дължина `k` са необходими `k + 1` байта памет.
- **Инициализация:** `char str[6] = "Hello";` (автоматично добавя `\0` на края).

Основни операции (библиотека `<cstring>`)

- `strlen(s)`: Връща броя на символите без терминиращия символ.
- `strcpy(dest, src)`: Копира съдържанието на един низ в друг. Програмистът трябва да осигури достатъчно място в целевия буфер.
- `strcmp(s1, s2)`: Лексикографско сравнение. Връща `0` при равенство, отрицателна стойност ако `s1 < s2` и положителна ако `s1 > s2`.
- `strcat(dest, src)`: Конкатенация (слепване) – добавя втория низ в края на първия.
- `strchr(s, c)` / `strstr(s, sub)`: Търсене на символ или подниз.

4. Рекурсия

Определение и структура

Рекурсията е метод, при който функцията извиква сама себе си пряко или косвено. Тя се състои от два задължителни елемента:

1. **База (дъно):** Най-простият случай, при който изчисленията спират. Без база рекурсията е безкрайна и води до препълване на стека (*stack overflow*).
2. **Рекурсивна стъпка:** Свеждане на по-сложната задача към по-проста версия на същата задача.

Видове рекурсия

- **Пряка:** Функция `f` извиква директно `f`.
- **Косвена:** Функция `f` извиква `g`, а `g` извиква `f`.
- **Линейна:** Всяко извикване прави най-много едно рекурсивно извикване (например Факториел).

    - *Опашкова рекурсия:* Рекурсивното извикване е последното действие във функцията (оптимизация).
- **Разклонена:** Едно извикване прави две или повече рекурсивни извиквания (например Числа на Фибоначи). Сложността често е експоненциална ($O(2^n)$), но може да се оптимизира чрез мемоизация (memoization).

Предимства и недостатъци

- **Предимства:** Силна изразителност, удобна за задачи от тип "разделяй и владей" и търсене с връщане назад (*backtracking* - например път в лабиринт).
- **Недостатъци:** Скрито използване на памет за стекови рамки (параметри, локални променливи, адрес за връщане), потенциална неефективност.

Глоссарий на ключовите термини

Термин	Дефиниция
Дереференциране	Процес на достъп до стойността, съхранявана на адреса, към който сочи даден указател.
Нулев терминатор (`\0`)	Символ с ASCII код 0, който отбелязва края на символния низ в C-стил.
Стекова рамка	Блок памет в програмния стек, съдържащ локални променливи и параметри на текущо изпълняваната функция.
Heap (Динамична памет)	Област от паметта, управлявана ръчно от програмиста чрез `new` и `delete`.
Memory Leak	Ситуация, при която динамично заделена памет не е освободена, но указателят към нея е изгубен.
Backtracking	Алгоритмичен метод за търсене на решение чрез проба, грешка и връщане към предходен избор.
Лексикографска наредба	Подредба на низове, подобна на подредбата в речник, базирана на ASCII стойностите на символите.

Мемоизация	Техника за оптимизация на рекурсия чрез запазване на резултатите от вече изчислени подзадачи.

13

13. Обектно-ориентирано програмиране

1. Абстракция и основи на класовете

Абстракция със структури от данни

Абстракцията е процес на разделяне на логическото описание (интерфейса) на даден тип данни от неговата физическа реализация.

- **Принцип:** Потребителят на структурата работи с операции (методи), без да е необходимо да познава вътрешното представяне на данните.
- **Нива на абстракция:** Обикновено включват физическо представяне (напр. `struct`), базови операции (конструктори, селектори) и приложни програми.
- **Предимства:** Изолиране на промените (промяна в реализацията не засяга горните нива) и ограничаване на нужните познания за работа с обекта.

Класове и обекти

- **Клас (Class):** Синтактична конструкция за дефиниране на абстрактни типове данни. Той е шаблон, който групира данни (член-данни) и операции (методи). В C++ `class` по подразбиране има `private` достъп, докато `struct` е `public`.
- **Обект (Object):** Конкретна инстанция на клас. Всеки обект заема определено количество памет (sizeof), като член-данните му се подравняват подобно на записите.

Декларация на клас и обект

```
class Rational {
private:
   int numer, denom; // член-данни
public:
   Rational();       // декларация на конструктор
   void read();      // декларация на метод
};

Rational r1;         // Декларация на обект (инстанция)
```

Основни видове конструктори

Конструкторите инициализират обекта и го поставят във валидно състояние. Те имат същото име като класа и нямат тип на връщане.

1. **Конструктор по подразбиране (Default):** Извиква се без аргументи. Ако не е дефиниран нито един конструктор, компилаторът генерира системен такъв.
2. **Конструктор с параметри:** Позволява задаване на начални стойности при създаване.
3. **Копиращ конструктор (Copy Constructor):** Създава нов обект като копие на съществуващ (`A(const A& other)`). Извиква се при предаване по стойност или връщане на обект от функция.

4. **Конструктор за местене (Move Constructor):** Ефективно прехвърля ресурси от временни обекти, избягвайки скъпо копиране.

Управление на динамичната памет и ресурсите (RAII)

RAII (Resource Acquisition Is Initialization) е ключов принцип, при който жизненият цикъл на ресурса (памет, файлове) е обвързан с жизнения цикъл на обекта:

- **Конструктор:** Заделя ресурса.
- **Деструктор:** Автоматично освобождава ресурса.
- **Правило на „Голямата тройка“:** Ако класът управлява динамична памет, той трябва експлицитно да дефинира: деструктор, копиращ конструктор и оператор за присвояване (`operator=`).

Методи – декларация, параметри и резултат

- **this указател:** Неявен параметър във всяка нестатична член-функция, който сочи към обекта, за който е извикана функцията.
- **Предаване на параметри:** Може да бъде по стойност, по указател или по референция (често `const&` за ефективност).
- **Константни методи:** Маркират се с `const` след параметрите. Те не могат да променят член-данните на обекта и са единствените методи, които могат да се викат за константни обекти.

2. Наследяване, капсулация и статични компоненти

Наследяване (Inheritance)

Наследяването моделира връзката **„е“ (is-a)**. Позволява създаването на нови класове (производни) на базата на съществуващи (основни).

- **Производен клас:** Получава всички член-данни и методи на основния клас (без конструкторите, деструктора и приятелите му).
- **Жизнен цикъл:** При създаване първо се извиква конструкторът на основния клас. При унищожаване деструкторите се викат в обратен ред (първо на производния).

Видове наследяване и достъп

Достъпът до наследените компоненти се определя от спецификаторите:

| Спецификатор | Достъп в самия клас | Достъп в наследници | Външен достъп |

| **public** | Да | Да | Да |

| **protected** | Да | Да | Не |

| **private** | Да | Не | Не |

Капсулация и скриване на информация

Капсулацията разделя интерфейса от имплементацията чрез спецификатори за достъп.

- **Цел:** Защита на инвариантите на класа (данните да не се променят неконтролирано).
- **Инструменти:** Член-данните обикновено са `private`, а достъпът до тях става чрез **селектори** (get-функции) и **мутатори** (set-функции с проверки за валидност).

Вложени класове и Композиция

- **Вложени класове:** Дефинират се в областта на видимост (scope) на друг клас. Използват се за логическо групиране на помощни типове.
- **Композиция (Composition):** Моделира връзка **„има“ (has-a)**. Един клас съдържа обект от друг клас като член-променлива.

Статични полета и методи

Статичните компоненти принадлежат на самия клас, а не на отделните обекти.

- **Статични член-данни:** Има само едно копие за целия клас, споделено от всички обекти. Изискват дефиниция извън класа.
- **Статични член-функции:** Могат да се извикват без създаден обект (`ClassName::Method()`). Те **нямат** достъп до `this` и могат да работят само със статични данни.

Глосарий на ключовите термини

- **Абстракция:** Скриване на детайлите по реализацията и предоставяне само на необходимия интерфейс.
- **Деструктор:** Специален метод, който се грижи за почистването на ресурсите при унищожаване на обекта.
- **Инвариант:** Условие, което винаги трябва да е вярно за състоянието на един коректен обект.
- **Капсулация:** Механизъм за ограничаване на директния достъп до данните на обекта.
- **Конструктор:** Метод за първоначално установяване на състоянието на обекта.
- **Мутатор (Setter):** Метод, който променя стойността на частна член-променлива под контрол.
- **Наследяване:** Механизъм за преизползване на код, при който един клас приема атрибутите и поведението на друг.
- **Обект:** Екземпляр на клас, притежаващ собствено състояние в паметта.
- **Селектор (Getter):** Метод за достъп до стойността на частна член-променлива без нейната промяна.
- **Статичен член:** Компонент (променлива или функция), който е общ за всички инстанции на класа.

14

14. Обектно-ориентирано програмиране: Полиморфизъм и наследяване

1. Виртуални функции и подтипов полиморфизъм

Същност на полиморфизма

Полиморфизмът е свойството на даден елемент да приема повече от една форма. В контекста на ООП това означава един и същ интерфейс или име на операция да работи с обекти от различни типове.

- **Подтипов полиморфизъм:** Позволява обекти от различни класове в една йерархия да се третират унифицирано чрез указател или препратка към техния базов клас.

Статично срещу динамично свързване

- **Статично свързване (early binding):** Изборът на функция се прави от компилатора по време на компилация (напр. претоварване на функции). В C++ това е поведението по подразбиране.
- **Динамично свързване (late binding):** Изборът на метод се определя по време на изпълнение в зависимост от реалния тип на обекта, към който сочи указателят. Постига се чрез **виртуални функции**.

Виртуални функции и `override`

Виртуалната функция се декларира с ключовата дума `virtual` в базовия клас.

- **Ключова дума `override`:** Указва, че методът в производния клас предефинира виртуален метод от базовия. Това позволява на компилатора да провери за съответствие в сигнатурите и предотвратява грешки.
- **Виртуални деструктори:** Задължително е деструкторът на базовия клас да бъде виртуален, ако класът ще се използва полиморфно. В противен случай, при изтриване на обект през указател към базовия клас, ще се извика само деструкторът на базовия клас, което води до изтичане на памет (memory leak).

Абстрактни методи и класове

- **Чисто виртуална функция (абстрактен метод):** Декларира се със синтаксиса `virtual void f() = 0;`. Тя няма реализация в базовия клас и задължава наследниците да я реализират.
- **Абстрактен клас:** Клас, който съдържа поне една чисто виртуална функция. От него не могат да се създават обекти; той служи единствено за интерфейс и наследяване.

Масиви от обекти и указатели (Хетерогенни контейнери)

- **Проблемът "Object Slicing" (Изрязване):** Ако създадем масив от обекти от базов тип и се опитаме да съхраним в него обекти от производни класове, данните от производната част се "изрязват" и се губи полиморфното поведение.
- **Хетерогенен контейнер:** Масив от указатели към базовия клас. Това позволява съхранение на обекти от различни типове (наследници) в един общ контейнер, като се запазва техният реален тип и се осигурява динамично свързване при извикване на методи.

2. Параметричен полиморфизъм (Шаблони)

Същност

Параметричният полиморфизъм позволява една и съща имплементация да работи с различни типове данни, като типът се предава като параметър. В C++ това се реализира чрез **шаблони (templates)**.

Шаблони на функции

Позволяват дефиниране на общи функции (например `swap`, `max`, `reverse`), които работят с произволен тип `T`, стига той да поддържа необходимите за тялото на функцията операции.

- Компилаторът генерира конкретен код за всеки използван тип (процес, наречен инстанциране).
- Типовите параметри могат да бъдат изведени автоматично или указани явно: `swap<int>(a, b)`.

Шаблони на класове

Позволяват създаването на генерични структури от данни (напр. `Vector<T>`, `Stack<T>`).

- Типовите параметри се използват за член-данни, параметри на методи и връщани резултати.
- **Особеност:** Шаблоните на класове не се компилират, докато не бъдат инстанцирани с конкретен тип.

Шаблонна специализация

Можем да дефинираме специална версия на шаблон за определен тип, ако стандартната имплементация не е ефективна или приложима (например специализация на `vector<bool>` за пестене на памет).

3. Множествено наследяване

Дефиниция

Множественото наследяване позволява на един производен клас да има повече от един директен базов клас: `class Bot : public Player, public AI`. Класът комбинира характеристиките и поведението на всички свои родители.

Проблеми при множественото наследяване

1. **Неоднозначност (Ambiguity):** Ако два базови класа имат методи с еднакви имена, извикването им през производния клас води до грешка при компилация. Решението е изрично указване на област: `obj.Base1::print()`.
2. **Диамантен проблем (Deadly Diamond of Death):** Възниква, когато два класа (B и C) наследяват от един и същ базов клас (A), а четвърти клас (D) наследява и от двата (B и C). Това води до дублиране на обекта от тип A в паметта на D.

Виртуално наследяване

За решаване на диамантения проблем се използва **виртуално наследяване**:

- Декларира се чрез `class B : virtual public A`.
- Гарантира, че в обекта от най-ниско ниво в йерархията ще има само едно споделено копие на базовия клас A.
- **Особеност на конструкторите:** При виртуално наследяване най-производният клас е отговорен за извикването на конструктора на виртуалния базов клас.

Речник на ключовите термини

Термин	Дефиниция
RTTI	(Run-Time Type Information) Механизъм за намиране на типа на обект по време на изпълнение (`typeid`).
VTable	(Виртуална таблица) Таблица с указатели към виртуалните функции на даден клас.
VPtr	Скрит указател в обекта, който сочи към неговата виртуална таблица.
Интерфейс	Абстрактен клас, който съдържа само чисто виртуални функции и няма член-данни.
Хетерогенен масив	Масив, съдържащ указатели към различни обекти от една йерархия (чрез базов указател).
Чисто виртуална функция	Функция без тяло в базовия клас, означена с `= 0`, която дефинира задължителен интерфейс.

Шаблон (Template)	Програмна конструкция, позволяваща генериране на код въз основа на параметризирани типове.
Prototype Pattern	Шаблон за дизайн, използващ метод `clone()` за копиране на обекти без знание за техния конкретен тип.

15

15. Структури от данни: Списък, Стек, Опашка и Дървовидни структури

1. Дефиниране на понятието „Структури от данни“

Структурите от данни представляват схема за организация, съхранение и управление на данни в паметта на компютъра, обикновено с цел постигане на ефективност при изпълнение на определени операции. Те се разделят на няколко основни категории:

- **Според вида на компонентите:**
  - *Хомогенни:* Съставени от елементи от един и същи тип (напр. масиви).
  - *Хетерогенни:* Съставени от елементи от различни типове (напр. структури или класове).
- **Според способността за промяна на размера:**
  - *Статични:* Размерът е фиксиран при компилиране (напр. статичен масив).
  - *Динамични:* Размерът може да се променя по време на изпълнение (напр. динамичен масив).
- **Според връзките между елементите:**
  - *Линейни:* Елементите са подредени последователно (списък, стек, опашка).
  - *Разклонени:* Елементите имат йерархична връзка (дърво).
  - *Мрежови:* Сложни връзки между елементите (граф).

2. Списък (List)

Списъкът е хомогенна линейна структура с последователен достъп до елементите. За разлика от масива, елементите на свързания списък не е задължително да заемат последователни адреси в паметта.

Логическо описание и реализации

1. **Едносвързан списък (Single Linked List):** Всеки възел съдържа данни и указател (`next`) към следващия елемент. Последната връзка сочи към `nullptr`. Обхождането е само в една посока.
2. **Двусвързан списък (Double Linked List):** Всеки възел съдържа данни и два указателя – към следващия (`next`) и към предходния (`prev`) елемент. Позволява двупосочно обхождане.

Сложност на операциите

Операция	Едносвързан списък	Двусвързан списък
`empty()` (проверка за празнота)	O(1)	O(1)

`push_front()` (добавяне в начало)	O(1)	O(1)
`push_back()` (добавяне в край)	O(1) (ако има указател към края)	O(1)
`pop_front()` (премахване от начало)	O(1)	O(1)
`pop_back()` (премахване от край)	O(n)	O(1)
`insert(x, p)` / `delete(p)` (позиция p)	O(n)	O(n)
`get(p)` / `find(x)` (търсене)	O(n)	O(n)

Дефиниране на клас (пример за едносвързан списък)

Класът трябва да съдържа вътрешна структура `Node` и указател към главата (`head`).

```
struct Node {
    int data;
    Node* next;
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() : head(nullptr) {}
    void push_front(int x);
    void pop_front();
    bool empty();
    // Деструктор за освобождаване на паметта
};
```

3. Стек (Stack)

Стекът е линейна структура, следваща принципа **LIFO** (*Last In, First Out*) – последният влязъл елемент излиза пръв.

Характеристики на реализациите

- **Статична реализация:** Използва масив с фиксиран размер. Бърза, но с ограничен капацитет. Операциите са O(1).
- **Динамична реализация (разширяващ се стек):** Използва динамичен масив. При препълване се заделя нова памет (обикновено двойно повече) и данните се копират. Амортизирана сложност на `push` е O(1).
- **Свързана реализация:** Използва едносвързан списък. Върхът на стека е главата на списъка. Предимство: няма ограничение в размера (освен паметта на компютъра). Недостатък: използва повече памет за указатели.

Сложност на операциите

Всички основни операции (`push`, `pop`, `peek/top`, `empty`) имат сложност **O(1)**.

Дефиниране на клас (свързана реализация)

```
struct StackElement {
    int data;
    StackElement* next;
};

class Stack {
private:
    StackElement* top;
public:
    Stack() : top(nullptr) {}
    void push(int x);
    int pop();
    int peek();
    bool empty();
};
```

4. Опашка (Queue)

Опашката е линейна структура, следваща принципа **FIFO** (*First In, First Out*) – първият влязъл елемент излиза пръв.

Характеристики на реализациите

- **Статична (циркулярна) реализация:** Използва масив и два индекса (`front` и `rear`). Чрез вземане на остатък по модул размерът на масива се „затваря“ в кръг, избягвайки ненужно местене на елементи. Сложност O(1).
- **Динамична реализация:** Динамичен цикличен масив, който се преоразмерява при нужда.
- **Свързана реализация:** Използва свързан списък с два указателя – към началото (`front`) за премахване и към края (`back`) за добавяне. И двете операции са O(1).

Сложност на операциите

Основните операции (`enqueue` – добавяне, `dequeue` – премахване, `front` – достъп) са със сложност **O(1)**.

5. Дървовидни структури от данни

Дървото е йерархична структура от възли, в която всеки елемент (освен корена) има точно един родител.

Логическо описание

- **Кореново дърво:** Състои се от корен (R) и множество поддървета.
- **Двоично дърво:** Всеки възел има най-много две деца (ляво и дясно).
- **Основни термини:** Лист (възел без деца), Височина (дължина на най-дългия път от корена до лист), Ниво (разстояние от корена).

Представяне в паметта

1. **Свързано представяне:** Всеки възел съдържа стойност и указатели към своите деца. При двоичните дървета – `left` и `right`.
2. **Последователно представяне (в масив):** Използва се основно за пълни двоични дървета. За възел на индекс i, лявото дете е на 2i+1, а дясното на 2i+2.

Обхождане на двоично дърво

- **Префиксно (Preorder):** Корен -> Ляво -> Дясно (КЛД)
- **Инфиксно (Inorder):** Ляво -> Корен -> Дясно (ЛКД)
- **Постфиксно (Postorder):** Ляво -> Дясно -> Корен (ЛДК)

6. Двоично кореново дърво за търсене (BST)

Двоичното дърво за търсене (Binary Search Tree) е организация, която поддържа наредба на елементите за бърз достъп.

Логическо описание

За всеки възел със стойност V:

- Всички стойности в **лявото** поддърво са **по-малки** от V.
- Всички стойности в **дясното** поддърво са **по-големи** от V.

Сложност на операциите

Сложността зависи от височината на дървото (h). При балансирано дърво с n елемента, h \approx \log(n).

| Операция | Среден случай (балансирано) | Най-лош случай (изродено в списък) |

| Търсене (`search`) | O(\log n) | O(n) |

| Добавяне (`insert`) | O(\log n) | O(n) |

| Премахване (`remove`) | O(\log n) | O(n) |

Забележка: Балансирани дървета като AVL или Червено-черни дървета гарантират сложност O(\log n) чрез автоматично преструктуриране.

Речник на ключовите термини

- **LIFO (Last In, First Out):** Принцип, при който последният добавен елемент е първият, който се извлича (характерен за стека).
- **FIFO (First In, First Out):** Принцип, при който първият добавен елемент е първият, който се извлича (характерен за опашката).
- **Възел (Node):** Основен градивен елемент на свързаните структури, съдържащ данни и връзки (указатели).
- **Корен (Root):** Най-горният възел в дървовидна структура, който няма родител.
- **Лист (Leaf):** Възел в дърво, който няма наследници (деца).
- **Сложност O(n):** Линейна сложност, при която времето за изпълнение нараства правопропорционално на броя на елементите.
- **Сложност O(log n):** Логаритмична сложност, характерна за операции в балансирани дървета и двоично търсене.
- **nullptr:** Специална стойност на указател в C++, обозначаваща, че той не сочи към валиден адрес в паметта (използва се за край на списък или празно дете в дърво).

16

16. Софтуерно инженерство: Процеси, модели и методологии

1. Понятие за софтуерно инженерство

Същност и обхват

Софтуерното инженерство е инженерна дисциплина, която прилага систематични, дисциплинирани и количествено измерими подходи към проектирането, разработката, поддръжката и управлението на софтуерни системи с високо качество. За разлика от индивидуалното програмиране, то обхваща целия жизнен цикъл на продукта — от първоначалната идея до извеждането му от употреба.

Софтуерът не е само програмен код. Той включва три основни компонента:

- **Инструкции:** Компютърни програми, които осигуряват желаната функционалност и производителност.
- **Структури от данни:** Позволяват на програмите да съхраняват и обработват информация ефективно.
- **Документация:** Описва работата и използването на програмите.

Ключови характеристики:

- Софтуерът се **разработва**, а не се произвежда в класическия смисъл.
- Той не се износва физически, но подлежи на **остаряване** поради технологични иновации и промени в бизнес нуждите.

Видове софтуерни продукти и системи

Софтуерните продукти се делят на две основни категории според техния целеви пазар:

1. **Общи (Generic):** Самостоятелни системи, предназначени за широкия пазар (напр. текстови редактори).
2. **Поръчани (Customized):** Разработени специално за нуждите на конкретен клиент.

Вид софтуер	Описание	Примери
Системен	Обслужва други програми; интензивна комуникация с хардуера.	ОС, драйвери, компилатори.
Приложен	Решава специфични нужди на потребителите.	Текстови редактори, плейъри.

Научен	Използва се за симулации, анализи и емпирично тестване.	Симулатори на климата, CAD системи.
Вграден (Embedded)	Интегриран в хардуерни устройства.	Софтуер в перални, автомобили, климатици.
Уеб приложения	Многослойни системи, достъпни през браузър.	Социални мрежи, онлайн магазини.
Изкуствен интелект	Използва нечислови алгоритми за сложни проблеми.	Разпознаване на образи, невронни мрежи.
Наследен (Legacy)	Стари системи с критична важност за бизнеса.	Банкови системи, разработени преди десетилетия.

2. Софтуерен процес: Фази и дейности

Софтуерният процес е последователност от предвидими стъпки и съвкупност от процедури, организирани за създаването на висококачествен продукт.

Основни фази на разработка

1. **Изисквания:** Събиране и анализ на нуждите на потребителите.
2. **Проектиране:** Архитектурно и детайлно планиране на системата.
3. **Реализация:** Самото програмиране (кодиране).
4. **Тестване:** Проверка на компонентите (Unit testing), интеграцията и цялата система.
5. **Внедряване:** Инсталиране и предоставяне на клиента.
6. **Поддръжка:** Корекции и актуализации след внедряването.

Универсални дейности в процеса

- **Комуникация:** Сътрудничество с клиента за изясняване на функционалността.
- **Планиране:** Дефиниране на ресурси, рискове и график.
- **Моделиране:** Създаване на технически спецификации чрез анализ и дизайн.
- **Конструиране:** Генериране на код и тестване.
- **Внедряване:** Оценка на продукта от крайния потребител.

Езици и шаблони за моделиране

За визуализация на системите най-често се използва **UML (Unified Modeling Language)**. Моделите могат да бъдат:

- **Описателни:** Как се разработва системата.

- **Предписателни:** Как би трябвало да се разработва.

Шаблонът на процес е описание на доказано решение на общ проблем. Всеки шаблон съдържа: контекст, проблем, решение, краен контекст и свързани шаблони.

3. Сравнителен анализ на описателни модели

Модел	Характеристика	Предимства	Недостатъци	Подходящ за...
Водопад (Waterfall)	Линеен и последователен подход.	Яснота, лесно управление, строга документация.	Негъвкав; късно откриване на грешки; изисква пълни изисквания в началото.	Проекти със стабилни и добре известни изисквания.
RAD (Бърза разработка)	Високоскоростен, модулен, паралелен (60-90 дни).	Бърз резултат чрез готови компоненти и автоматичен код.	Не е за сложни архитектури; изисква модулност.	Малки до средни проекти с ясни модули.
Прототипен	Създаване на ранна версия (еволюционна или изхвърляема).	Помага за изясняване на изискванията; ранна обратна връзка.	Може да изразходва много ресурси; риск от лоша архитектура.	Проекти с неясни потребителски изисквания.
Спираловиден	Итеративен модел с фокус върху **анализа на риска**.	Гъвкав; подходящ за промени; систематично управление на риска.	Скъп; сложен за управление; изисква експерти по риска.	Големи, сложни и високорискови проекти.
Постъпков (Incremental)	Системата се изгражда и доставя на части (инкременти).	Ранна доставка на ядрото; по-лесна адаптация.	Изисква много добра първоначална архитектура.	Системи, които трябва бързо да пуснат базова функция.

4. Гъвкави методи за разработка (Agile)

Agile методите поставят акцент върху хората, работещия софтуер и адаптацията към промени, вместо върху строгото следване на планове и документация.

Scrum

Scrum организира работата в кратки итерации, наречени **спринтове** (1-4 седмици).

- **Роли:** Product Owner (представител на бизнеса), Scrum Master (фасилитатор) и Development Team (самоорганизиращ се екип).
- **Артефакти:** Product Backlog (списък с всички нужди) и Sprint Backlog (задачи за текущия спринт).
- **Събития:** Планиране, Ежедневна среща (Daily Scrum), Преглед на спринта (Review) и Ретроспектива.

Extreme Programming (XP)

XP се фокусира върху техническото качество чрез специфични практики:

- **Чифтно програмиране:** Двама души работят на един компютър.
- **TDD (Test-Driven Development):** Първо се пише тест, после минималният код.
- **Непрекъсната интеграция:** Често обединяване на програмния код.
- **Рефакториране:** Подобряване на вътрешната структура на кода без промяна на поведението му.

5. Концепция за многократна употреба (Reusability)

Практиката за използване на съществуващи компоненти, библиотеки и рамки (frameworks) за пестене на време и повишаване на качеството.

- **COTS (Commercial Off-The-Shelf):** Готови софтуерни продукти, които предлагат API (напр. платежни системи като Stripe).
- **Компонентно-базирана разработка:** Сглобяване на системи от независими единици с дефинирани интерфейси (**Requires** - какво му трябва, **Provides** - какво дава).
- **Шаблони за проектиране (Design Patterns):** Абстрактни решения на архитектурни проблеми (Създаващи, Структурни, Поведенчески).

Глосарий на ключовите термини

- **UML (Unified Modeling Language):** Стандартизиран език за моделиране, използван за визуализация, спецификация и документиране на софтуерни системи.
- **Инкремент (Increment):** Работеща, тествана и потенциално доставима част от софтуера, добавена към предишни версии.
- **Backlog (Product Backlog):** Приоритизиран списък с всички функционалности, подобрения и корекции, които трябва да бъдат направени в продукта.
- **Валидация:** Процес на проверка дали софтуерът отговаря на реалните нужди и изисквания на потребителя.
- **Верификация:** Процес на проверка дали софтуерът е разработен правилно спрямо техническата спецификация.
- **Вграден софтуер (Embedded Software):** Софтуер, който е част от хардуерно устройство и управлява неговите функции.

- **Scrum Master:** Роля в Scrum, отговорна за подпомагане на екипа, премахване на пречки и гарантиране на правилното прилагане на методологията.
- **TDD (Test-Driven Development):** Методология за разработка, при която тестовете се пишат преди самия функционален код.
- **Интерфейс (Interface):** Точка на взаимодействие между различни компоненти или системи; дефинира какви услуги се предоставят или изискват.
- **CASE инструменти (Computer-Aided Software Engineering):** Софтуерни средства, които подпомагат и автоматизират дейностите по разработване и поддръжка на софтуер.

17

17. Модели на разпределени софтуерни архитектури

1. Параметри на паралелната и разпределена обработка, метрика, методи за анализ

Паралелната обработка включва едновременно изпълнение на данни от множество „работници“ (процеси или нишки). Тя става разпределена, когато тези работници са разположени на различни физически машини (изчислителни възли).

- **Основни параметри:**
  - **Паралелизъм (p):** Броят работници. Софтуерният паралелизъм (брой стартирани нишки) е ограничен от хардуерния (брой ядра/машини).
  - **Грануларност:** Степен на декомпозиция на задачата. Бива едра (малко подзадачи, лесно разпределение, лош баланс), фина (много подзадачи, добър баланс, голям овърхед) и средна (оптимален компромис).
  - **Балансиране:** Целта е всеки процес да получи равно натоварване. Може да бъде статично (предварително) или динамично (по време на изпълнение). Структурите биват централизирани (звезда), разпределени (решетка, хиперкуб) или йерархични.
  - **Синхронизация:** Асинхронна (без обмен на данни), локално синхронна (комуникация със съседи) и глобално синхронна (пълен граф на комуникация).
  - **Декомпозиция:** По данни (SPMD – една програма върху различни данни) или по управление (MPMD – различни програми).
- **Метрики и аномалии:**
  - **Ускорение ($S_p = T_1 / T_p$):** Обикновено е между 1 и p.
  - **Ефикасност ($E_p = S_p / p$):** Измерва се в проценти (0 до 1).
  - **Суперлинейна аномалия ($S_p > p$):** Възниква при оптимизация на работата с паметта (кеша) при паралелно изпълнение.
  - **Нелинейна (немонотонна) аномалия:** Забавяне при увеличаване на паралелизма ($S_{p+1} < S_p$) поради комуникационни закъснения или хардуерни ограничения.

2. Модели на разпределените софтуерни архитектури

- **Процедурни:** Базирани на отдалечено извикване на процедури (RPC) или методи (RMI). Прозрачни за извикващия възел.
- **Обектни:** Използват абстракция, композиция, наследственост, енкапсулация и полиморфизъм. Сътрудничество между независими обекти.
- **Потокови (Flow):** Системата е поредица от трансформации.
  - *Пакетна обработка:* Модули в последователен ред (Batch sequential).
  - *Филтриране и канали (Pipe and Filter):* Конвейерно изпълнение, интерактивност.
  - *Контролни процеси:* Реално време (напр. в авиацията).
- **Контекстни (Data Centric):** Централизирано хранилище.
  - *Repository:* Активни агенти извличат данни (СУБД, Web-услуги).
  - *Blackboard:* Агентите са абонати, реагиращи на промени (реактивни).
- **Йерархични:**
  - *Слоест модел:* Слоеве (OSI, TCP/IP), всеки предоставя услуга на горния.
  - *Master/Slaves:* Главен модул разпределя работата за отказоустойчивост и баланс.
- **Асинхронни:** Message passing.

    - *Producer/Consumer (1:1):* Интерактивен (online) или отложен (offline).
    - *Publish/Subscriber (1:много):* Базиран на теми (topic-based) или контекст.
- **Интерактивни:** Интензивен потребителски интерфейс. Модели MVC (Model-View-Controller) и PAC. Декомпозиция на модули за представяне, данни и управление.

3. Организация на разпределените приложения

- **Клиент-Сървър:** Класически MPMD модел. Клиентите биват „тънки“ (thin) или „дебели“ (fat).
- **Многослойни модели:**
    - *Двуслойни:* Клиент (UI) и сървър (логика и данни).
    - *Трислойни:* Разделяне на Presentation Layer, Application Layer (бизнес логика) и Data Layer.
    - *N-слойни:* Повече слоеве за услуги, интеграция и сигурност.
- **Peer-to-Peer (P2P):** Равноправни участници, децентрализация и скалируемост.
    - *Неструктурирани:* Произволна топология.
    - *Структурирани:* Изрична топология (напр. хорда), използващи DHT.
    - *Хибридни:* Използват супервъзли.
- **Специализирани среди:**
    - *Сървъри за приложения:* Управляват бизнес логика (Tomcat, JBoss).
    - *Web-сървъри:* HTTP съдържание (Apache, Nginx).
    - *Метасистеми и Грид:* Обединяване на ресурси за сложни научни задачи.
    - *SOA и Микроуслуги:* Услуги като градивни блокове с дефинирани интерфейси.
    - *Софтуерни агенти:* Автономни, адаптивни компоненти, вземащи самостоятелни решения.

Терминологичен речник

Термин	Дефиниция
SPMD	(Single Program, Multiple Data) Модел на декомпозиция, при който една и съща програма се стартира върху различни данни.
MPMD	(Multiple Program, Multiple Data) Модел, при който се използват няколко различни програми за управление на задачата.
RPC / RMI	Механизми за отдалечено извикване на процедури или методи, осигуряващи прозрачност в процедурните модели.
MVC	(Model-View-Controller) Интерактивен архитектурен модел, разделящ системата на данни, изглед и логика на управление.

DHT	(Distributed Hash Tables) Технология, използвана в структурираните P2P мрежи за ефективно адресиране и търсене на ресурси.
SOA	(Service-Oriented Architecture) Архитектура, базирана на независими услуги, комуникиращи чрез стандартни протоколи.
Латентност	Времето за предаване на съобщение от източника до получателя в разпределена система.
Микроуслуги	Архитектурен стил, при който приложението е набор от малки, автономни услуги със специфични бизнес функции.
Грид (Grid)	Специализирана метасистема, обединяваща разпръснати компютри за решаване на обща сложна задача.
Енкапсулация	Обектно-ориентиран принцип за скриване на вътрешните детайли на обекта от външния свят.

18

18. Модели и методи за проектиране на потребителски интерфейс

1. Основни модели и методи при създаване на потребителски интерфейс

Потребителският интерфейс е средството за комуникация между човека и компютъра. Проектирането му е мултидисциплинарен процес, обединяващ компютърни науки, психология, социология и инженерни науки.

Основни подходи

- **User-Centered Design (Проектиране, ориентирано към потребителя):** Потребителят е в центъра на процеса; интерфейсът се адаптира към неговите нужди и контекст.
- **Итеративен дизайн:** Непрекъснато усъвършенстване чрез цикли на проектиране, тестване и оценка.
- **Гъвкави методологии (Agile):** Бърза адаптация и подобрения въз основа на обратна връзка.

Итеративен процес на разработка

1. **Идентифициране на нужди и изисквания:** Събиране на информация чрез интервюта и анкети.
2. **Концептуален дизайн:** Изясняване на структурата и организацията на информацията.
3. **Макети и прототипи:** Създаване на ниско детайлни (wireframes) и интерактивни прототипи (напр. във Figma).
4. **Визуален дизайн:** Определяне на цветови схеми, шрифтове и графични елементи.
5. **Тестване и оценка:** Провеждане на тестове за използваемост с реални потребители.

Анализ на задачите

Методите за анализ помагат за дефиниране на действията на потребителя:

- **Task Analysis:** Общ анализ на задачите.
- **Cognitive Task Analysis:** Фокусира се върху умствената дейност (решаване на проблеми, внимание).
- **Hierarchical Task Analysis (HTA):** Представя действията като йерархия от прости стъпки.

Инструментални средства: Figma, Sketch, Adobe XD, Powerpoint, UML, LucidChart.

2. Проектиране на графичен интерфейс (GUI)

Графичният интерфейс включва визуални елементи като прозорци, икони, менюта, графики и бутони.

Интерактивни стилове

- **Директно въвеждане (Direct manipulation):** Работа директно с обекти (напр. влачене на файл в кошчето).
- **Избор от меню:** Избор на команди от списък.
- **Попълване на форми:** Въвеждане на данни в структурирани полета.
- **Команден език:** Ръчно въвеждане на текстови команди.
- **Естествен език:** Взаимодействие чрез говор или чат (напр. Siri).

Психологични аспекти и концептуални модели

- **Когнитивна психология:** Отчитане на ограниченията на кратковременната памет и вниманието.
- **Гещалт принципи:** Използване на закони за близост и сходство за групиране на елементи.
- **Метафори:** Използване на познати концепции от реалния свят (напр. „десктоп“, „кошче“, „ракета“ за излитане/изпращане).

Реализация на интерфейси

- **Сценарии (Storyboarding):** Описват последователност от стъпки в определена среда.
- **Хартиени прототипи:** Бързи и евтини скици за ранна обратна връзка.
- **Wizard of Oz:** Техника, при която човек скрито симулира реакциите на системата (backend-а).

3. Разработка на използваем графичен интерфейс: Експериментални техники

Оценката на използваемостта (Usability) се извършва в контролирана среда или реални условия чрез следните техники:

Метод	Описание
Think Aloud	Потребителите разсъждават на глас, докато работят със системата.
A/B Testing	Сравняване на две версии на интерфейса за ефективност.
Eye Tracking	Анализ на визуалните маршрути (накъде гледа потребителят).
Инжектиране на код	Софтуерно събиране на данни за грешки и най-използвани елементи.

Usability Testing	Директно наблюдение и видеозапис на работата на потребителите.

4. Разработка на мултимедиен графичен интерфейс

Цветове и модели

Цветовете се използват за привличане на внимание и обратна връзка.

- **RGB:** Червено, зелено, синьо (0-255). За екрани.
- **CMYK:** Циан, магента, жълто, черно (%). За печат.
- **HSL:** Оттенък, наситеност, светлина.
- **Схеми:** Монохроматични, аналогични, допълващи и др. Обикновено се избират три цвята: главен, второстепенен и акцентен (tertiary).

Графика: Растерна срещу Векторна

Характеристика	Растерна графика	Векторна графика
Състав	Матрица от пиксели	Математически уравнения (линии, криви)
Скалиране	Губи качество при увеличение	Без загуба на качество
Формати	jpg, png, gif	SVG, EPS, AI
Приложение	Снимки, сложни детайли	Лога, икони, илюстрации

Текст, звук и анимация

- **Текст:** Използване на UTF-8 кодиране; форматиране на големи блокове за четливост.
- **Звук:** Формати mp3, wav, ogg за подобряване на изживяването.
- **Анимация:** Формати mov, mp4, avi. Подпомага разбирането на взаимодействията.

---

5. Особености при създаване на интегриран интерфейс

Интегрираният интерфейс изисква фокус върху крайния потребител и консистентност.

Методи за моделиране

- **Етнография:** Наблюдение на потребителите в тяхната естествена ежедневна среда.
- **Контекстен дизайн:** Вникване в специфичния домейн на потребителя (напр. банкова сфера).
- **Метод на участието:** Потребителят е част от екипа по разработка.

Принципи на екранния дизайн и иконите

- **Икони:** Трябва да са ясно видими, различими една от друга и сходни по стил, без излишни детайли.
- **Обработка на взаимодействията:** Интерфейсът трябва да дава смислени съобщения за грешки, индикация за успешно завършена команда и лесен достъп до помощна информация.

Глосарий на ключовите термини

- **A/B Testing:** Експериментален метод за сравняване на две версии на дизайн с цел определяне на по-ефективната.
- **Eye Tracking:** Технология за проследяване на движението на очите на потребителя върху екрана.
- **User-Centered Design (UCD):** Проектиране, при което нуждите, желанията и ограниченията на крайния потребител са водещи.
- **Wireframe:** Ниско детайлен макет на интерфейс, фокусиран върху разположението на елементите, а не върху визуалния стил.
- **Директна манипулация (Direct Manipulation):** Стил на взаимодействие, при който потребителят действа директно върху визуалните обекти (напр. преместване на икона).
- **Етнография:** Метод на изследване чрез пряко наблюдение на потребителите в тяхната реална работна или жизнена среда.
- **Итеративен процес:** Повтарящ се цикъл от фази, целящ постепенно подобряване на крайния продукт.
- **Когнитивна психология:** Наука за умствените процеси, използвана в дизайна за разбиране на начина, по който хората възприемат и помнят информация.
- **Прототип:** Ранен модел на продукт, създаден за тестване на концепции и събиране на обратна връзка.
- **Уеб достъпност/Използваемост (Usability):** Степента, до която един интерфейс е лесен за научаване и ефективен за използване.

19

19. Управление на качеството на софтуерни приложения и тестване на софтуер

1. Осигуряване на качеството

Осигуряване на качеството (Quality Assurance - QA) е съвкупност от дейности, проектирани да оценят процеса, чрез който се създават и поддържат софтуерните продукти. Неговата основна цел е да докаже правилното поведение на системата и да установи механизми за откриване и отстраняване на проблеми.

Качество на софтуера

Качеството се дефинира като степента, до която системата или нейните компоненти отговарят на специфицираните изисквания и удовлетворяват очакванията на заинтересованите лица. То може да се разглежда от две гледни точки:

- **Съответствие с изискванията:** Реализиране на описаните функционални и нефункционални параметри.
- **Удовлетворяване на потребителските нужди:** По-широк поглед, включващ полезност и надеждност в конкретен контекст, дори ако първоначалните изисквания са били непълни.

Алтернативи за осигуряване на качеството

1. **Ориентирана към процеса (Софтуерна разработка):** Фокус върху превенцията на дефекти чрез ясни изисквания, стандарти за кодиране и ревюта.
2. **Ориентирана към продукта (Софтуерна поддръжка):** Фокус върху динамичното тестване и поддръжката на вече готовия продукт.

2. Тестови дейности, управление и автоматизация

Тестването е процес на изпълнение на софтуера с цел откриване на дефекти. Важно е да се отбележи, че тестването доказва наличието на грешки, но не може да гарантира тяхното пълно отсъствие.

Основни тестови дейности

- **Планиране на тестовете:** Дефиниране на цели, стратегия, ресурси и критерии за край.
- **Конструиране на тестов модел:** Анализ на информацията и създаване на модел за тестване.
- **Генериране на тестови сценарии (Test cases):** Колекция от входове и очаквани резултати.
- **Управление на тестови пакети (Test suites):** Групиране на сценарии, които се изпълняват в последователност.
- **Изпълнение и наблюдение:** Стартиране на тестовете и събиране на резултати чрез "тестови оракули".

- **Анализ и проследяване:** Идентифициране на повреди и управление на дефекти (регистрация и приоритизация).

Управление на екипите

- **Вертикален модел:** Организацията е около конкретен продукт.
- **Хоризонтален модел:** Един екип извършва специфичен тип тестване за всички продукти в голяма организация.
- **Смесен модел:** Ниските нива на тестване са към проектите, а системното тестване се споделя.

Автоматизация на тестването

Целта е освобождаване на тестерите от повтарящи се задачи (например регресионно тестване). Автоматизацията има висока цена за поддръжка и не заменя човешката преценка при изследователско тестване.

Тестова дейност	**Възможност за автоматизация**
Изпълнение на тестове	Висока
Анализ на надеждността и покритието	Висока
Генериране на тестови сценарии	Висока
Създаване на формални модели	Средна
Цялостно планиране и подобряване на продукта	Ниска

3. Видове тестване

Тестване с контролни списъци (Checklists)

Това е надграждане на **Ad-hoc тестването** (свободно наблюдение). Използват се неформални списъци ("TODO") за проследяване. Видовете списъци биват базови, йерархични, комбинирани и смесени. Основен проблем е трудността при описване на сложни взаимодействия.

Тестване с класове на еквивалентност

Входният домейн се разделя на групи (класове), за които се очаква системата да се държи идентично. Избира се по един представител от всеки клас.

- **Валидни класове:** Данни, които системата трябва да приеме.
- **Невалидни класове:** Данни, които трябва да предизвикат грешка или отказ.

Разделяне на входния домейн и тестване на границите

- **Разделяне на домейна:** Систематичен избор на стойности чрез идентифициране на входното пространство и поддомейни.
- **Тестване на границите (Boundary Value Analysis):** Фокусира се върху стойностите "на ръба" (например <, <=), където най-често възникват програмни грешки.

Други техники

- **Тестване по двойки (Pairwise testing):** Използване на ортогонални масиви за намаляване броя на тестовете при много комбинации.
- **Таблици на решенията:** Използват се при сложна логика с комбинации от условия (True/False) и съответни действия.

4. Нива на тестване и приложение на техниките

Тестовете са организирани йерархично според нивото на абстракция:

1. **Тестване на ниво програмна единица (Unit testing):** Най-ниското ниво. Проверява отделни функции или класове. Извършва се от разработчици чрез метода на "бялата кутия".
2. **Интеграционно тестване (Integration testing):** Проверява взаимодействията между модулите. Използва се както "бяла", така и "черна кутия".
3. **Системно тестване (System testing):** Тестване на цялата система като едно цяло по метода на "черната кутия" срещу функционалните изисквания.
4. **Тестване за приемане (Acceptance testing):** Проверка на готовността за реална експлоатация. Включва Алфа (в контролирана среда) и Бета тестване (от реални потребители).

5. Измерване и метрики

Измерването е процес на съпоставяне на стойности към характеристиките на обекта.

Характеристики на метриките

- **Надеждност:** Еднозначни резултати при еднакви условия.
- **Валидност:** Измерва точно това, което се изисква.
- **Практичност:** Лесно събиране и анализ на данните.

Класификация на метриките

- **Процесни метрики:** Ефективност на разработката (напр. брой открити дефекти на етап, средно време за изпълнение).
- **Продуктови метрики:** Качество на софтуера (напр. MTTF – средно време до отказ, MTTR – средно време за възстановяване, процент грешки, удовлетвореност на потребителите).
- **Проектни метрики:** Управление на ресурси, време и разходи.

Глосар на ключовите термини

- **Ad-hoc тестване:** Неформален подход на тестване чрез стартиране на софтуера и свободно наблюдение на неговото поведение.
- **Unit testing (Модулно тестване):** Тестване на най-малките програмни единици (функции, методи, класове) в изолация.
- **Алфа тестване:** Тестване на продукта в контролирана среда от вътрешни екипи преди предаването му на клиентите.
- **Бета тестване:** Тестване от ограничена група реални потребители в тяхната собствена работна среда.
- **Измерване:** Процес на съпоставяне на стойности към характеристиките на изследван обект по определени правила.
- **Метрика:** Функция, която съпоставя състоянията на дадена характеристика на нейните мерки (количествен показател).
- **Осигуряване на качеството (QA):** Превантивни дейности, насочени към подобряване на процесите на разработка.
- **Регресионно тестване:** Повторно тестване на вече проверена функционалност, за да се гарантира, че новите промени не са внесли дефекти.
- **Тестов оракул:** Механизъм или източник на информация за проверка дали резултатът от теста е правилен.
- **Тестова серия (Test run):** Динамична единица, представляваща изпълнението на един или няколко теста върху избран обект.

20

20. Софтуерна архитектура: Проектиране и документиране

1. Понятие за софтуерна архитектура

Дефиниция и същност

Софтуерната архитектура на една система представлява съвкупност от структури, които показват различните софтуерни елементи, техните външно видими свойства и връзките помежду им. Тя е **абстракция**, която разглежда елементите като „черни кутии“, фокусирайки се върху техните взаимодействия, а не върху вътрешната им реализация (алгоритми и детайли).

Структури и изгледи (Structures and Views)

Важно е да се прави разлика между структура и изглед:

- **Структура:** Реалната съвкупност от софтуерни елементи и техните връзки.
- **Изглед (View):** Конкретно документирано представяне на дадена структура, предназначено за комуникация със заинтересованите лица.

Основни групи архитектурни структури:

1. **Модулни структури:** Описват функционалните компоненти (например модули за продукти, плащания, потребители).
2. **Структури на процесите:** Описват динамичното поведение и комуникацията по време на изпълнение (например как заявка преминава през слоевете).
3. **Структури на разположението (Физически):** Показват разпределението на софтуерните компоненти върху хардуерни ресурси (сървъри, бази данни, кеш сървъри).

Основни архитектурни изгледи:

- **Логически изглед:** Абстракции като обекти и класове.
- **Изглед на процесите:** Взаимодействащи си процеси в реално време.
- **Изглед на кода:** Организация на файловете и пакетите.
- **Физически изглед:** Разположение върху хардуерни възли.

Необходимост и влияние

Архитектурата е необходима за управление на сложността в големи системи. Тя осигурява обща картина и език за комуникация между разработчици, тестери и мениджъри.

- **Влияние върху проекта:** Определя решения, които са трудни за промяна и насочват всички последващи действия.
- **Влияние върху организацията:** Структурата на екипите често следва архитектурата на системата (например отделни екипи за различни услуги или слоеве).

2. Архитектурни стилове

Архитектурните стилове са общи шаблони за организация, които влияят пряко върху качествените характеристики на системата.

Стил	Описание	Предимства/Недостатъци
Многослоен (Layered)	Вертикално разделени слоеве. Всеки слой предлага услуги на горния и ползва долния.	Ясно разделение на отговорностите; риск от намалена производителност.
MVC (Model-View-Controller)	Разделя данните (Модел), представянето (Изглед) и логиката на управление (Контролер).	Позволява една логика да има различни интерфейси; улеснява тестването.
Поточен (Pipe-and-filter)	Последователност от филтри, свързани чрез канали (pipes). Всеки филтър трансформира данни.	Възможност за паралелизъм и повторна употреба; разходи за парсване на данни.
Ориентиран към услуги (SOA)	Система от самостоятелни услуги със строго дефинирани интерфейси (договори).	Гъвкавост и слаба свързаност; сложност при мрежова комуникация.
Клиент-сървър	Сървър предлага ресурси, а клиентът ги консумира.	Класическо разпределение на отговорностите.
Shared-data	Данните служат като конектор. Вариации: Repository и Blackboard.	Централизирано управление на състоянието.

3. Проектиране на софтуерната архитектура

Процес на проектиране

Проектирането е **цикличен процес**, който започва с анализ на нуждите на заинтересованите лица (stakeholders).

1. **Идентифициране на драйвери:** Избор на най-приоритетните функционални изисквания (use-cases) и сценарии за качество.
2. **Декомпозиция:** Разделяне на системата йерархично на модули.
3. **Избор на тактики и шаблони:** Прилагане на конкретни решения за постигане на драйверите.
4. **Валидация:** Проверка дали декомпозицията отговаря на изискванията.

Сценарии за качество

Качеството се формализира чрез сценарии, състоящи се от 6 компонента:

- **Контекст:** Обстоятелства (например "по време на работа").
- **Източник:** Кой инициира събитието (например "потребител").
- **Въздействие (Стимул):** Какво се случва (например "прави справка").
- **Обект:** Коя част от системата е засегната.
- **Резултат:** Как реагира системата.
- **Количествени параметри:** Измерими критерии (например "за по-малко от 15 сек").

Тактики за качествени показатели

Тактиките са конкретни технически решения:

- **Надеждност:** Ping/echo, Heartbeat, активен/пасивен излишък (redundancy).
- **Производителност:** Кеширане, Load-balancing (разпределение на натоварването), паралелна обработка.
- **Сигурност:** Криптиране, хеширане на пароли, двуфакторна автентикация (2FA).
- **Скалируемост:** Хоризонтално (scaling out) и вертикално (scaling up) скалиране, sharding на бази данни.
- **Изменяемост:** Прилагане на принципа Open-Closed.

4. Документиране на софтуерната архитектура

Предназначение и принципи

Документацията служи за комуникация, поддръжка, обучение на нови кадри и основа за разработка.

- **Основен принцип:** Документацията трябва да е съобразена с нейните читатели. Тя трябва да е точна, актуална и да обяснява не само „какво“, но и „защо“ е взето дадено решение.

Съдържание и структура на документацията

Един стандартен архитектурен документ включва:

1. **Въведение:** Цели и обхват.
2. **Архитектурни изгледи:** Детайлни описания (логически, физически и др.).
3. **Сценарии за качество:** Описание на ключовите нефункционални изисквания.
4. **Архитектурни решения:** Обосновка на избраните стилове и тактики (включително разгледани алтернативи).
5. **Елементи и връзки:** Детайлно описание на интерфейсите.
6. **Терминологичен речник:** Дефиниции за избягване на двусмислие.

Глосарий на ключовите термини

- **Архитектурен драйвер (Architectural Driver):** Изискване (функционално или качествено), което има значително влияние върху архитектурните решения.
- **Изглед на разположението (Deployment View):** Архитектурен изглед, показващ как софтуерните компоненти са разпределени върху хардуерната инфраструктура.

- **Интерфейс:** Външно видимите свойства на един елемент, които му позволяват да взаимодейства с други елементи; „договор“ за комуникация.
- **Кеширане (Caching):** Тактика за производителност, при която често използвани данни се съхраняват на достъпно място за бързо извличане.
- **Конвейеризация (Pipelining):** Метод в Pipe-and-filter архитектурата, позволяващ паралелна обработка на различни части от данните.
- **Сценарий за качество (Quality Scenario):** Метод за специфициране на нефункционални изисквания чрез описание на конкретна ситуация и очаквана реакция на системата.
- **Тактика:** Конкретно проектно решение, насочено към постигане на определен качествен атрибут (напр. репликация за надеждност).
- **Черна кутия (Black Box):** Концепция в архитектурата, при която се разглеждат само входните и изходните сигнали на един елемент, без да се познава неговата вътрешна логика.

21. Инженеринг на софтуерните изисквания

1. Същност на Инженеринга на софтуерните изисквания

Инженерингът на изискванията е систематичен процес на **идентифициране, анализиране, документиране и валидиране** на функционалностите и ограниченията на даден софтуер. Докато програмирането отговаря на въпроса „как“, инженеринг на изискванията отговаря на въпроса **„каква система трябва да се реализира“**.

Цели и задачи:

- Идентифициране и решаване на проблеми.
- Дефиниране на обхвата на системата.
- Свързване на различните заинтересовани лица (ЗЛ) – клиенти, потребители, разработчици; те идентифицират проблеми които се нуждаят от решения
- Осигуряване на основа за проектиране, тестване и приемане на продукта.

Основни етапи на процеса:

1. **Извличане (Elicitation):** Търсене и откриване на изисквания от ЗЛ и други източници.
2. **Анализ и преговори:** Уточняване на изискванията и решаване на конфликти между тях.
3. **Документиране (Специфициране):** Записване на изискванията в структуриран документ.
4. **Валидиране:** Проверка дали изискванията точно отразяват нуждите на клиента.
5. **Управление:** Паралелна дейност за контрол на промените и проследимост.

2. Класификация на софтуерните изисквания

Изискванията се разделят на няколко основни категории, за да се улесни тяхното управление и реализация:

Категория	Описание	Примери
Функционални (ФИ)	Какво трябва да прави системата (действия, функции).	„Системата позволява регистрация на потребител.“
Нефункционални (НФИ)	Качествени аспекти и ограничения (как работи системата).	Производителност, сигурност, надеждност, използваемост.
Бизнес изисквания	Високо ниво на цели на организацията.	„Намаляване на времето за обработка на поръчки.“

Системни изисквания	Подробни технически спецификации и зависимости.	Хардуерни ограничения, софтуерни зависимости.
Домейн изисквания	Произтичат от конкретната приложна област.	Банкови регулации, физични закони, медицински стандарти.

3. Техники за извличане, анализ и валидиране

Извличане на изискванията

- **Интервюта и анкети:** Пряка комуникация със ЗЛ за разбиране на техните нужди.
- **Сценарии и Случаи на употреба (Use Cases):** Примери от реалния живот за взаимодействие със системата.
- **Наблюдение (Етнография):** Анализ на потребителите в реалната им работна среда.
- **Прототипиране:** Създаване на ранна версия на системата за събиране на обратна връзка.
- **Мозъчна атака (Brainstorming):** Екипна работа за генериране на идеи и алтернативи.

Анализ на изискванията

- **Проверка за необходимост и осъществимост:** Реалистични ли са изискванията спрямо бюджета и технологиите?
- **Матрица на зависимостите:** Откриване на конфликти и припокривания между изискванията.
- **Приоритизиране:** Използване на методи като **MoSCoW** (Must, Should, Could, Won't).

Валидиране на изискванията

- **Прегледи (Reviews):** Систематичен ръчен анализ на документацията.
- **Генериране на тестови случаи:** Ако едно изискване не може да бъде тествано, то е лошо дефинирано.

4. Специфициране и модели на изискванията

Документирането превръща събраната информация в техническа спецификация. Моделите помагат за визуализиране на системата от различни перспективи:

- **Модел на потока на данните (DFM/DFD):** Показва как информацията се движи и трансформира в системата. Елементи: Процеси, Външни субекти, Хранилища на данни.
- **Модел на поведението (STD):** Описва състоянията на системата и преходите между тях в отговор на събития.
- **Семантичен модел (ERD):** Представя обектите (същностите) в базата данни, техните атрибути и връзки.
- **Обектно-ориентирани модели (Клас диаграми):** Обединяват данни и функционалност чрез обекти, класове и наследяване.
- **Формални модели:** Използват математическа нотация за максимална точност (синтаксис, семантика, връзки).

Глоссарий на ключовите термини

- **Заинтересовани лица (ЗЛ / Stakeholders):** Всички субекти (клиенти, потребители, експерти), които имат интерес от системата или влияние върху нейните изисквания.
- **Случай на употреба (Use Case):** Техника за описване на взаимодействието между актьор (потребител/система) и софтуерната система за постигане на конкретна цел.
- **Спецификация на софтуерните изисквания (SRS):** Официален документ, който описва пълния набор от функции и ограничения на системата.
- **Валидиране (Validation):** Процес на потвърждение, че изискванията съответстват на реалните нужди на клиента („Правилната система ли строим?“).
- **Проследимост (Traceability):** Способността да се проследи жизнения цикъл на едно изискване от неговия произход до съответния дизайн, код и тест.
- **CASE инструменти:** Автоматизирани софтуерни инструменти, подпомагащи анализа и моделирането на изискванията.
- **Капсулиране:** Концепция от обектно-ориентираните модели, при която данните и операциите върху тях са обединени в един обект.
- **Кардиналност:** Свойство на семантичните модели (ERD), описващо количествената връзка между същностите (например 1:1, 1:N).
- **Преговори за изискванията:** Процес на обсъждане и постигане на компромис при наличие на конфликтни изисквания от различни ЗЛ.

22

22. Проектиране и интегриране на софтуерни системи

1. Характеристики на разпределените софтуерни системи

Дефиниция и същност

Разпределената софтуерна система се дефинира като група от независими компютри, които за крайния потребител изглеждат като една цялостна, логически единна система. Тези компютри са свързани в мрежа, а техните хардуерни и софтуерни компоненти координират действията си единствено чрез обмен на съобщения.

Основното предизвикателство при проектирането е постигането на координация и консистентност в среда, където компонентите са разделени по пространство и време. Целите на тези системи са висока производителност, надеждност, мащабируемост и лесна поддръжка.

Основни характеристики

- **Прозрачност (Transparency):** Стремеж за скриване на сложните детайли на разпределението. Включва:
    - *Прозрачност на местоположението:* Достъп до ресурс без знание за физическото му разположение.
    - *Прозрачност на достъпа:* Еднакъв интерфейс за локални и отдалечени ресурси.
    - *Прозрачност на репликацията:* Скриване на наличието на множество копия данни.
    - *Прозрачност на отказите:* Прикриване на повреди или автоматично възстановяване.
- **Хетерогенност:** Обединяване на компоненти с различни технологии, платформи и езици чрез стандартизирани протоколи (HTTP, JSON, XML).
- **Мащабируемост:** Способност за обработка на нарастващо натоварване чрез хоризонтално (добавяне на възли) или вертикално (увеличаване ресурсите на възел) мащабиране.
- **Отказоустойчивост:** Проектиране на системата така, че да продължи работа при отказ на отделни компоненти чрез репликация и резервиране.

Видове разпределени системи

1. **Разпределени изчислителни системи:** Използват се за задачи с висока производителност.
    - *Клъстер:* Идентични възли с еднаква ОС, физически близки, работещи паралелно.
    - *Грид:* Хетерогенни, често физически отдалечени възли с различен хардуер и ОС.
2. **Разпределени информационни системи:** Свързват оперативно несъвместими приложения.
    - *Клиент-сървър (Client-Server):* Клиентите пращат заявки, сървърът ги обработва. Възможна е атомарна обработка на групирани заявки.
    - *Peer-to-Peer (P2P):* Всеки възел е едновременно клиент и сървър; директна комуникация.
3. **Широкоразпространени (Pervasive) системи:** Интегрират малки и мобилни устройства (телефони, умни часовници, IoT устройства).

Тенденции

- Разширяване на Интернет като глобална разпределена система.
- Навлизане на облачни изчисления (Cloud computing) – AWS, Azure, Google Cloud.
- Увеличено търсене на мултимедийни услуги и системи по заявка (on-demand).
- Интеграция на Интернет на нещата (IoT).

2. Междупроцесна комуникация (IPC)

Комуникацията между процесите се осъществява чрез обмен на съобщения (Inter Process Communication). Операционните системи предоставят механизми като опашки от съобщения (message queues), семафори и споделена памет (shared memory). Често тези механизми се използват чрез **middleware** (междинен софтуер), който предоставя абстракция чрез интерфейси.

Видове комуникация

- **Connection-oriented:** Изгражда се специален канал за комуникация.
- **Message-oriented:** Преизползване на съществуващи канали.

Отдалечено извикване

Начин процес A да използва функционалност на процес B. Двете основни технологии са:

1. **RPC (Remote Procedure Call):** Базира се на интерфейси, дефинирани чрез **IDL (Interface Definition Language)**. Скрива реализацията на модула.
    - *Семантики на RPC:*
        - *Maybe:* Изпълнява се веднъж или нито веднъж.
        - *At-least-once:* Изпълнява се поне веднъж; клиентът получава потвърждение или информация за липса на резултат.
        - *At-most-once:* Изпълнява се най-много веднъж.
    - *gRPC:* Модерна реализация от Google.
2. **RMI (Remote Method Invocation):** Обектно-ориентиран подход, позволяващ предаване на референции към обекти.
    - *Компоненти на RMI:*
        - *Proxy/Stub (Клиент):* Препраща заявките към отдалечения обект.
        - *Skeleton (Сървър):* Имплементира интерфейса и делегира работата на Servant.
        - *Dispatcher:* Получава заявки и извиква съответния метод на Skeleton.
        - *Remote Reference Module:* Управлява съпоставката на локални и отдалечени референции.
        - *Servant:* Самата инстанция на отдалечения обект.

Мултикаст (Multicast)

Метод за едновременно изпращане на съобщения до група получатели (видео стрийминг, онлайн игри).

- **Gossip техника:** Оптимизация на разпространението, при която всеки "заразен" възел предава информацията на съседите си (Gossip-Based Data Dissemination).

3. Разпределени обекти и компоненти

Разпределени обекти

Прилагат обектно-ориентирания модел в разпределена среда. Комуникацията се извършва чрез RMI или разпределени събития. Обектите могат да са на различни машини, но си взаимодействат чрез извикване на методи.

Разпределени компоненти

Софтуерният компонент е по-едра градивна единица от обекта. За него са дефинирани предлагани интерфейси и контекстни зависимости. Компонентите могат да се внедряват и тестват независимо.

- **Примери:** Enterprise Java Beans (EJB) и CORBA (Common Object Request Broker Architecture).

4. Уеб услуги и стандарти

Уеб услугите предоставят функционалност чрез стандартизирани интернет протоколи (HTTP). Те абстрахират изпълнението от конкретните програмни езици. Използват XML или JSON за обмен на данни.

Шаблони за комуникация

- **Синхронна (Request-Response):** Клиентът блокира и изчаква отговор.
- **Асинхронна:** Клиентът не се блокира.
- **Базирана на събития (Publish-Subscribe):** Получаване на съобщения при настъпване на събитие (напр. webhooks).
- **One-way:** Изпращане на съобщение без очакване на отговор.

Основни стандарти

1. **SOAP (Simple Object Access Protocol):** XML-базиран протокол над HTTP/SMTP. Поддържа сигурност и транзакции.
    - *Структура на съобщението:* Envelop (обвивка), Header (допълнителна информация), Body (основни данни).
2. **WSDL (Web Service Description Language):** XML език за описание на интерфейса на услугата (методи, параметри, типове данни).
3. **UDDI (Universal Description, Discovery and Integration):** Регистър за откриване на услуги (аналогично на телефонен указател).
    - *Бели страници:* Информация за доставчици.
    - *Жълти страници:* Класификация на услугите.
    - *Зелени страници:* Техническа информация.

Терминологичен речник

Термин	Дефиниция

Middleware	Междинен софтуер, който служи като обвивка около комуникационните механизми на ОС и предоставя абстракция чрез интерфейси.
IDL (Interface Definition Language)	Език за дефиниране на интерфейси, чрез които се специфицират процедурите, предлагани от отдалечен процес в RPC.
Gossip Algorithm	Техника за разпространение на информация в мрежа, при която всеки възел предава данни на съседите си, подобно на разпространение на вирус.
Servant	Конкретна инстанция на отдалечен обект в RMI модела, която изпълнява заявените операции.
WSDL	XML-базиран език, използван за описание на интерфейса, методите и параметрите на една уеб услуга.
SOAP	Протокол за обмен на структурирана информация под формата на XML обекти в разпределена среда.
Мащабируемост	Способността на системата да се справя с по-голямо натоварване чрез добавяне на допълнителни ресурси.
Хетерогенност	Свойство на системата да включва компоненти, изградени с различни програмни езици, операционни системи и хардуер.
Skeleton	Компонент на сървърната страна в RMI, който имплементира отдалечения интерфейс и предава работата на Servant.
IoT (Internet of Things)	Концепция за свързване на множество разнообразни физически устройства (сензори, уреди) към интернет в рамките на разпределена система.

23

23. Планиране на проекта: Същност, обхват, ресурси и управление

1. Планиране на проекта – същност и основни елементи

Същност на проекта и планирането

Проектът е ограничено от времето и ресурсите начинание, чиято цел е създаването на конкретен и уникален резултат (продукт или услуга). Основните характеристики на проекта включват:

- **Време за изпълнение (Temporality):** Има дефинирано начало и край.
- **Единственост на резултата (Deliverables):** Създава се нов продукт, знание или документ.
- **Прогресивно развитие (Progressive elaboration):** Развива се постепенно чрез итерации.

Планът на проекта е основният управленски документ, който определя как целите ще бъдат постигнати, как ще се изпълнява работата и как ще се упражнява контрол. Основната му цел е да намали неопределеността и да координира участниците.

Основни елементи на проектния план

Дейностите по планиране включват интегрирането на няколко подплана (често означавани с абревиатурата SSCQ – SCRP):

1. **Обхват и мисия (Scope):** Изясняване на изискванията и границите на проекта.
2. **График (Schedule):** Дефиниране на контролни точки и времева рамка.
3. **Разходи (Cost):** Апроксимация на необходимите финансови средства.
4. **Качество (Quality):** Стратегия за гарантиране на стандартите.
5. **Подобряване на процесите:** Оптимизация на дейностите.
6. **Човешки ресурси (Staffing):** Определяне на екипа и неговата наличност.
7. **Комуникации:** Управление на вътрешния и външния информационен поток.
8. **Риск:** Идентификация и оценка на потенциални заплахи.
9. **Поръчки и доставки (Procurement):** Управление на външни ресурси и договори.

2. Планиране на обхвата и Структура на работа (WBS)

Обхват на продукта срещу Обхват на проекта

- **Обхват на продукта:** Характеристиките и функциите на крайния резултат.
- **Обхват на проекта:** Работата, която трябва да бъде свършена, за да се достави продуктът с неговите характеристики.

Процеси по управление на обхвата

Планирането започва със **събиране на изисквания** чрез интервюта, фокус групи, мозъчни атаки, въпросници, наблюдения и прототипи. Изходът от този процес включва документация на изискванията и матрица за тяхното проследяване.

Дефинирането на обхвата използва техниката **DECAA**:

- **D**escription (Описание): Подробно описание на резултатите и критериите за приемане.
- **E**xclusions (Изключения): Какво НЕ е част от проекта.
- **C**onstraints (Ограничения): Бюджет, крайни срокове, договори.
- **A**ssumptions (Предположения): Проектни предложения и тяхното въздействие.
- **A**ctualisation (Актуализация): Модифициране на съпътстващите документи.

Йерархична структура на работа (WBS)

WBS е процес на разделяне на проекта на по-малки, лесноуправляеми компоненти чрез декомпозиция.

- **Работен пакет (Work Package):** Най-ниското ниво на WBS. Той съдържа описание на дейностите, зависимости, критерии за приемане, информация за изпълнителя, оценка на ресурсите и срок.
- **Метод на създаване:** Избира се абстрактен резултат и се разбива рекурсивно на подрезултати, докато се достигне ниво на конкретен работен пакет.

План на контролните точки (Milestones)

Контролните точки маркират началото и края на главни фази. Те служат за комуникация и оценка на напредъка.

- **Видове:** Основни (фази на жизнения цикъл), Второстепенни (завършване на сложна дейност), Отчетни (регулярни събития).

3. Време, ресурси, бюджет и дейности по контрол

Планиране на времето

Процесът включва:

1. Дефиниране и последователност на дейностите.
2. Оценка на ресурсите и продължителността.
3. Разработване и контролиране на графика.

Планиране на ресурси и бюджет

Ресурсите се делят на **материални** (инфраструктура, техника) и **нематериални** (човешки капитал). **Последователност за ценообразуване на работен пакет:**

1. Идентифициране на нужните материални и човешки ресурси.
2. Оценка на времето за различни конфигурации от ресурси.
3. Изчисляване на цената (ресурс × продължителност).
4. Приоритизиране и избор на оптимална конфигурация.

Бюджетът е финансовият план на проекта. Неговите компоненти включват разходи за персонал, оборудване, пътни разходи и подизпълнители. Управлението на бюджета включва оценка на разходите, бюджетиране и постоянен контрол.

Дейности по управление и контрол

- **Мониторинг:** Следене на реалното изпълнение спрямо плана.
- **Управление на промените:** Обработка на промени в обхвата, времето или бюджета.
- **Отчитане:** Редовни доклади към заинтересованите страни.
- **Решаване на проблеми:** Идентифициране на възникнали пречки по време на работа.

4. Създаване на план-график: Методи и средства

Стъпки при създаване на график

1. Определяне на предварително времетраене на задачите.
2. Създаване на идеален график според последователността.
3. Идентификация на ресурсите и реалното работно време.
4. Определяне на реално време (съобразено с празници и календари).
5. Модификация на основния план-график.

Методи за планиране

Метод	Описание	Ключови характеристики
Метод на критичния път (CPM)	Определя най-дългата последователност от задачи без резерв от време.	Закъснение в критична задача води до закъснение на целия проект.
Метод PERT	Използва се при неопределеност за изчисляване на очакваното време.	Формула: (T_{opt} + 4 \times T_{prob} + T_{pess}) / 6
GANTT диаграма	Визуален инструмент, представящ задачите като правоъгълници във времето.	Най-популярният метод за проследяване на прогреса и статуса.

Метод на критичния път (Параметри):

- **Early Start (ES) / Early Finish (EF):** Най-ранно възможно начало и край.
- **Late Start (LS) / Late Finish (LF):** Най-късно възможно начало и край без забавяне на проекта.
- **Float (Slack):** Резерв от време (LS - ES или LF - EF). При критичния път резервът е 0.

7. Глосар на ключовите термини

- **WBS (Work Breakdown Structure):** Йерархична структура за декомпозиция на работата по проекта на по-малки компоненти.
- **Scope Creep (Разширяване на обхвата):** Неконтролирано добавяне на нови изисквания без съответни промени в ресурсите и времето.

- **Критичен път (CPM):** Пътят в мрежовата диаграма с нулев резерв от време; определя минималния срок за завършване.
- **PERT (Program Evaluation and Review Technique):** Статистически метод за оценка на времетраенето при наличие на несигурност.
- **Работен пакет (Work Package):** Дефинирана задача в WBS, която може да бъде оценена, възложена и контролирана.
- **GANTT Диаграма:** Графично представяне на графика на проекта, показващо задачите като ленти по времева ос.
- **Float (Slack):** Времето, с което една задача може да бъде отложена, без това да повлияе на крайната дата на проекта.
- **Декомпозиция:** Техника за разбиване на абстрактни проектни резултати на детайлни и управляеми части.
- **ЗС (Заинтересовани страни):** Лица или организации, които имат интерес от резултата на проекта или са повлияни от него.

24

24. Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание

1. Добре-структуриран XML: Основни концепции и синтаксис

XML е език за маркиране, предназначен за създаване на структурирани документи, четими както от хора, така и от машини. Основната му цел е универсално съхранение и обмен на данни, независимо от платформата.

Основни компоненти

Всеки XML документ е изграден от следните градивни елементи:

- **Таг (Маркер):** Име на елемент, заградено в `<` и `>`.
- **Елемент:** Съвкупност от отварящ таг, съдържание и затварящ таг (напр. `<name>Ivan</name>`). Елементите могат да бъдат влагани, но винаги трябва да има точно един **коренов елемент** (root element).
- **Атрибут:** Допълнителна информация за елемента под формата на двойка `име="стойност"`, разположена в отварящия таг.
- **Метаданни:** Описание на семантиката на съдържанието.

XML йерархии

Структурата на XML се представя като йерархия (дърво), която може да бъде:

1. **Планарна:** Вертикално представяне на подредбата чрез влагане (nesting).
2. **Дървовидна:** Визуализира релациите „родител-дете“, където коренът е в основата, а листата са най-вложените елементи без под-елементи.

Синтактични правила

За да бъде един документ „добре-структуриран“ (well-formed), той трябва да спазва следните правила:

- **Чувствителност към регистъра:** `<Person>` и `<person>` са различни елементи.
- **Правилно затваряне:** Всеки отварящ таг задължително има съответстващ затварящ таг.
- **Без припокриване:** Елементите трябва да се затварят в реда, в който са отворени (аналогично на балансирани скоби).
- **Именуване:** Имената не могат да започват с „xml“ (в какъвто и да е регистър) или цифри. Могат да започват с буква или долна черта `_`.
- **Специални символи:** Символите `<`, `>`, `&`, `"` и `'` са резервирани и трябва да се заменят с техните entity референции (напр. `<` за `<`).

XML пространства от имена (Namespaces)

Използват се за избягване на колизии (конфликти) при еднакви имена на елементи от различни контексти. Дефинират се чрез атрибута `xmlns` и префикс, свързан с уникален URL адрес.

2. XML валидация чрез Document Type Definitions (DTD)

Валидацията проверява дали XML документът отговаря на предварително дефиниран речник и структура. DTD е традиционният механизъм за тази цел.

Цели и структура

DTD описва позволените елементи, техния ред и атрибути. То може да бъде **вътрешно** (inline в XML файла) или **външно** (в отделен `.dtd` файл).

Синтаксис и декларации

- **ELEMENT:** Дефинира модела на съдържание:
  - `EMPTY` – празен елемент.
  - `ANY` – произволно съдържание.
  - `#PCDATA` – текстово съдържание (Parsed Character Data).
  - **Индикатори за брой:** `+` (1 или повече), `*` (0 или повече), `?` (0 или 1).
- **ATTLIST:** Декларира атрибути за даден елемент. Типовете включват:
  - `CDATA` – обикновен текст.
  - `ID` – уникален ключ за елемента.
  - `IDREFS` – списък от референции към други ID-та.
  - **Декларация на стойност:** `#REQUIRED` (задължителен), `#IMPLIED` (опционален), `#FIXED` (фиксиран).
- **ENTITY:** Използва се като указател към текст или външен ресурс (напр. изображение).
- **NOTATION:** Специфицира типа на не-XML данни (напр. `image/jpeg`).

3. XML валидация чрез XML Schema (XSD)

XML Schema е по-мощен наследник на DTD, който сам по себе си е XML документ.

Спецификации и типове данни

За разлика от DTD, XML Schema поддържа богата палитра от типове данни (integer, date, boolean, string и др.). Тя разделя компонентите на:

- **SimpleType:** Елементи, които съдържат само текст и нямат атрибути или под-елементи.
- **ComplexType:** Елементи, които могат да съдържат под-елементи и атрибути.

Модели на групиране

- `<xs:sequence>`: Елементите трябва да се появят в точно указания ред.
- `<xs:all>`: Елементите могат да са във всеки ред, но трябва да присъстват.
- `<xs:choice>`: Позволява избор само на един елемент от списъка.

Фасети (Facets)

Фасетите са ограничения върху пространството от стойности на даден тип.

- **Ограничаващи:** `minInclusive/maxInclusive`, `length`, `pattern` (регулярни изрази), `enumeration` (изброяване).

Сравнение между DTD и XML Schema

Характеристика	DTD	XML Schema (XSD)
Синтаксис	Специфичен (не е XML)	XML синтаксис
Типове данни	Не поддържа (само текст)	Поддържа вградени и потребителски типове
Пространства от имена	Ограничена поддръжка	Пълна поддръжка
Мощност	Базова структура	Висока (фасети, наследяване)

4. XSLT и XPath: Трансформация и навигация

XPath (XML Path Language)

XPath служи за адресиране и извличане на части от XML документ, представяйки го като дърво от възли (корен, елемент, атрибут, текст).

- **Синтаксис:** Подобен на файлова система (`/` за корен, `//` за избор навсякъде).
- **Оси (Axes):** `child`, `parent`, `ancestor`, `descendant`, `attribute` (`@`).
- **Контекстен възел:** Отбелязва се с точка `.` (текущото местоположение на процесора).

XSLT (eXtensible StyleSheet Language Transformations)

XSLT се използва за трансформиране на XML в други формати (HTML, обикновен текст или друг XML).

- **Шаблони:** Използва се `<xsl:template match="...">` за дефиниране на правила за обработка.
- **Елементи за контрол:** `<xsl:for-each>` (цикъл), `<xsl:value-of>` (извличане на стойност), `<xsl:if>` (условие), `<xsl:variable>` (дефиниране на променливи).

5. Обработка на XML документи: DOM и SAX

За програмна работа с XML се използват два основни модела (API).

DOM (Document Object Model)

DOM зарежда целия XML документ в паметта като йерархична структура от обекти (`Node`).

- **Предимства:** Позволява произволен достъп, лесна навигация в двете посоки и модификация (добавяне/изтриване на елементи).
- **Недостатъци:** Консумира много памет при големи документи.
- **Основни интерфейси:** `Node`, `Document`, `Element`, `Attr`, `Text`.

SAX (Simple API for XML)

SAX е събитийно-ориентиран (event-driven) модел, който чете документа последователно (push модел).

- **Механизъм:** Парсерът генерира събития (напр. "старт на елемент", "текст"), които се обработват от манипулатори (`handlers`).
- **Предимства:** Много ниска консумация на памет; бърз при четене на огромни файлове.
- **Недостатъци:** Само четене (без модификация), навигация само напред.
- **Основни интерфейси:** `ContentHandler`, `XMLReader`, `ErrorHandler`.

Речник на ключовите термини

- **#PCDATA (Parsed Character Data):** Текстово съдържание в XML, което ще бъде анализирано от парсера.
- **CDATA (Character Data):** Област от данни, която парсерът не интерпретира (използва се за код или текст със специални символи).
- **Entity (Единица):** Променлива, използвана за заместване на повтарящ се текст или външни ресурси.
- **Facet (Фасет):** Ограничение в XML Schema, дефиниращо допустимите стойности за даден тип (напр. дължина).
- **Namespace (Пространство от имена):** Механизъм за уникално идентифициране на таговете чрез URI, предотвратяващ конфликти на имена.
- **Node (Възел):** Основна единица в дървовидната структура на DOM или XPath.
- **Root Element (Коренов елемент):** Най-външният елемент в XML документа, който съдържа всички останали.
- **XSD (XML Schema Definition):** Документ, дефиниращ структурата, съдържанието и ограниченията на XML инстанциите.
- **XSLT Template:** Правило в XSLT, което казва на процесора какво да прави при намиране на съвпадение с определен XPath път.

25

25. Теорема на Ферма, теореми за средните стойности и формула на Тейлър

1. Локален екстремум и теорема на Ферма

Дефиниция за локален екстремум

Нека функцията f е дефинирана в околност на точката c. Казваме, че f има **локален екстремум** в c, ако съществува интервал I около c, такъв че за всяко x \in I е изпълнено:

- f(x) \leq f(c) (локален максимум);
- f(x) \geq f(c) (локален минимум).

Това определение е локално, тъй като сравнява стойността в точката с само със стойностите в нейната непосредствена близост.

Теорема на Ферма (Необходимо условие за локален екстремум)

Ако функцията f е диференцируема в отворения интервал (a, b) и достига локален екстремум в точка c \in (a, b), то производната ѝ в тази точка е равна на нула: f'(c) = 0

Доказателство (за случай на максимум):

1. Тъй като c е вътрешна точка, можем да конструираме две редици (x_n) и (y_n), които клонят към c, като x_n < c < y_n.
2. От определението за максимум следва, че f(y_n) - f(c) \leq 0. Тъй като y_n - c > 0, то частното \frac{f(y_n) - f(c)}{y_n - c} \leq 0. При граница n \to \infty, получаваме f'(c) \leq 0.
3. За редицата отляво: f(x_n) - f(c) \leq 0. Тъй като x_n - c < 0, то частното \frac{f(x_n) - f(c)}{x_n - c} \geq 0 (защото и числителят, и знаменателят са отрицателни). При граница n \to \infty, получаваме f'(c) \geq 0.
4. Единствената възможност, удовлетворяваща f'(c) \leq 0 и f'(c) \geq 0, е f'(c) = 0.

2. Теореми за средните стойности

За установяването на тези теореми се използва **Теоремата на Вайерщрас**: всяка непрекъсната функция върху краен затворен интервал [a, b] достига своя абсолютен максимум и абсолютен минимум.

а) Теорема на Рол

Нека функцията f е:

1. Непрекъсната в затворения интервал [a, b].
2. Диференцируема в отворения интервал (a, b).
3. f(a) = f(b).

Тогава съществува поне една точка c \in (a, b), за която f'(c) = 0.

Доказателство: От теоремата на Вайерщрас f има максимум M и минимум m в [a, b].

- Ако M = m, функцията е константа и f'(x) = 0 за всяко x.
- Ако M \neq m, поне една от тези стойности се достига във вътрешна точка c \in (a, b) (тъй като f(a) = f(b)). По теоремата на Ферма, f'(c) = 0.

б) Теорема на Лагранж (Теорема за крайните нараствания)

Нека f е непрекъсната в [a, b] и диференцируема в (a, b). Тогава съществува c \in (a, b), такова че: f(b) - f(a) = f'(c)(b - a)

Доказателство: Дефинира се помощна функция d(x), представляваща разликата между f(x) и уравнението на правата (секущата), минаваща през (a, f(a)) и (b, f(b)): d(x) = f(x) - \left[\frac{f(b) - f(a)}{b - a}(x - a) + f(a) \right] Тъй като d(a) = d(b) = 0, прилагаме теоремата на Рол за d(x). Съществува c, за което d'(c) = 0, което води директно до формулата на Лагранж.

в) Теорема на Коши (Обобщена теорема за средните стойности)

Нека f и g са непрекъснати в [a, b] и диференцируеми в (a, b). Ако g'(x) \neq 0 за всяко x \in (a, b), то съществува c \in (a, b), за което: \frac{f(b) - f(a)}{g(b) - g(a)} = \frac{f'(c)}{g'(c)}

Доказателство: Първо се доказва, че g(b) \neq g(a) (чрез допускане на противното и теоремата на Рол). След това се прилага теоремата на Рол за функцията: h(x) = [f(b) - f(a)]g(x) - [g(b) - g(a)]f(x) Проверява се, че h(a) = h(b). От h'(c) = 0 следва търсеното съотношение.

3. Формула на Тейлър

Идеята на формулата е да се апроксимира (приближи) функция f(x) в околност на точка x_0 чрез полином от степен n.

Общ вид на формулата

f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k + R_n(x) Където:

- \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k е **полиномът на Тейлър**.
- R_n(x) е **остатъчен член**, представляващ грешката на приближението.

Остатъчен член във формата на Лагранж

Ако функцията притежава производни до ред (n+1), остатъчният член може да се запише като: R_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!}(x - x_0)^{n+1} за някое c между x_0 и x.

Извеждане: Използва се помощна функция \phi(t) и се прилага теоремата на Коши за интервала [x_0, x] спрямо функциите \phi(t) и \psi(t) = (x - t)^p. Чрез полагане на p = n + 1, се получава формата на Лагранж.

5. Речник на ключовите термини

Термин	Дефиниция
Производна	Границата на отношението на нарастването на функцията към нарастването на нейния аргумент при \Delta x \to 0.
Локален екстремум	Точка, в която стойността на функцията е най-голяма или най-малка в определена малка околност.
Стационарна точка	Точка c, в която f'(c) = 0.
Полином на Тейлър	Полином, чиито стойности и производни до определен ред съвпадат с тези на дадена функция в определена точка.
Остатъчен член	Разликата между реалната стойност на функцията и стойността, получена чрез полинома на Тейлър.
Теорема на Вайерщрас	Гарантира, че непрекъсната функция в затворен интервал задължително достига своите макс. и мин. стойности.
Фундаментална система решения (ФСР)	Базис на пространството от решения на хомогенна система линейни уравнения.

26. Определен интеграл

1. Основи на разбиването и суми на Дарбу

Дефиниция на разбиване на интервал

Нека разглеждаме затворен интервал [a, b], където a < b. **Разбиване P** на интервала се нарича крайна система от точки \{x_i\}_{i=0}^n, такива че: a = x_0 < x_1 < x_2 < \dots < x_{n-1} < x_n = b
Това разбиване разделя интервала на n подинтервала от вида [x_{i-1}, x_i]. Дължината на всеки подинтервал се бележи с \Delta x_i = x_i - x_{i-1}.

Диаметър (норма) на разбиването (\delta_P) е дължината на най-дългия подинтервал в дадено разбиване: \delta_P = \max_{1 \le i \le n} \Delta x_i

Суми на Дарбу

Нека функцията f(x) е дефинирана и ограничена в интервала [a, b]. За всеки подинтервал [x_{i-1}, x_i] дефинираме:

- **Инфимум (m_i):** най-голямата долна граница на стойностите на функцията в подинтервала.
- **Супремум (M_i):** най-малката горна граница на стойностите на функцията в подинтервала.

Въз основа на тях дефинираме двете суми на Дарбу:

1. **Малка сума на Дарбу (s):** \sum_{i=1}^n m_i \Delta x_i (геометрично съответства на лицето на „вписаната“ стъпаловидна фигура).
2. **Голяма сума на Дарбу (S):** \sum_{i=1}^n M_i \Delta x_i (геометрично съответства на лицето на „описаната“ стъпаловидна фигура).

Винаги е изпълнено неравенството: s \le S.

Ефект от добавяне на нови точки (Изфиняване на разбиването)

Когато към дадено разбиване P се добавят нови точки (създава се по-фино разбиване Q), се наблюдават следните промени:

- **Малките суми на Дарбу не намаляват (s_Q \ge s_P):** При разделяне на един подинтервал на две, инфимумът в новите части може само да се увеличи или да остане същият спрямо оригиналния инфимум.
- **Големите суми на Дарбу не нарастват (S_Q \le S_P):** Супремумът в по-малките части може само да намалее или да остане същият.

Геометрична интерпретация: При добавяне на точки, вписаната фигура се „качва нагоре“ и по-добре запълва пространството под графиката, а описаната фигура „слиза надолу“, прилепвайки по-плътно към нея.

2. Дефиниция на Риманов интеграл и критерий за интегруемост

Дефиниция чрез подхода на Дарбу

Разглеждаме колекцията \mathcal{P} от всички възможни разбивания на [a, b]:

- **Долен интеграл (L(f)):** супремумът на всички малки суми на Дарбу (L(f) = \sup \{s(P) : P \in \mathcal{P}\}).
- **Горен интеграл (U(f)):** инфимумът на всички големи суми на Дарбу (U(f) = \inf \{S(P) : P \in \mathcal{P}\}).

Ограничената функция f(x) е **интегруема по Риман**, ако долният и горният интеграл съвпадат: L(f) = U(f) = \int_a^b f(x) dx

Критерий за интегруемост

Ограничена функция f е интегруема върху [a, b] тогава и само тогава, когато за всяко \epsilon > 0 съществува разбиване P_\epsilon, такова че: S(P_\epsilon) - s(P_\epsilon) < \epsilon **Доказателство (идея):** Ако разликата може да стане произволно малка, това означава, че няма „разкъсване“ между горната и долната оценка на площта, следователно супремумът на малките и инфимумът на големите суми трябва да съвпаднат.

3. Интегруемост на непрекъснатите функции

Теорема: Всяка непрекъсната функция в краен и затворен интервал [a, b] е интегруема по Риман.

Доказателство:

1. **Равномерна непрекъснатост:** Съгласно теоремата на Кантор, всяка непрекъсната функция в затворен интервал е равномерно непрекъсната. Това означава, че за всяко \epsilon > 0 съществува \eta > 0, такова че ако |x - y| < \eta, то |f(x) - f(y)| < \frac{\epsilon}{b-a}.
2. **Избор на разбиване:** Избираме разбиване P с норма \delta_P < \eta.
3. **Оценка на разликата:** Във всеки подинтервал [x_{k-1}, x_k], разликата между максималната стойност M_k и минималната стойност m_k е по-малка от \frac{\epsilon}{b-a} (тъй като разстоянието между кои да е две точки в подинтервала е по-малко от \eta).
4. **Заключение:** S(P) - s(P) = \sum (M_k - m_k)\Delta x_k < \frac{\epsilon}{b-a} \sum \Delta x_k = \frac{\epsilon}{b-a} (b-a) = \epsilon Следователно, по критерия за интегруемост, функцията е интегруема.

4. Свойства на определения интеграл

Свойство	Описание (Формален запис)
Линейност	\int_a^b (\alpha f + \beta g)dx = \alpha \int_a^b f dx + \beta \int_a^b g dx

Адитивност	\int_a^b f dx = \int_a^c f dx + \int_c^b f dx за c \in [a, b]
Монотонност	Ако f(x) \le g(x), то \int_a^b f dx \le \int_a^b g dx
Интегриране на неравенства	Ако m \le f(x) \le M, то m(b-a) \le \int_a^b f dx \le M(b-a)
Смяна на границите	\int_a^b f dx = - \int_b^a f dx
Интеграл с еднакви граници	\int_a^a f dx = 0

5. Теореми за средната стойност и Нютон-Лайбниц

Теорема за средната стойност

Ако f(x) е непрекъсната в [a, b], то съществува c \in [a, b], за което: \int_a^b f(x) dx = f(c)(b-a)
Доказателство: От свойството за интегриране на неравенства имаме m \le \frac{1}{b-a} \int_a^b f dx \le M, където m и M са минимумът и максимумът на f. Тъй като непрекъснатата функция приема всички стойности между m и M (теорема на Болцано), съществува точка c, в която f(c) е равно на тази средна стойност.

Теорема на Нютон-Лайбниц

Тази теорема свързва определения интеграл с примитивната функция.

1. **Първа част:** Ако f е непрекъсната, то функцията F(x) = \int_a^x f(t)dt е диференцируема и F'(x) = f(x).
    - *Доказателство:* Използва се диферентното частно \frac{F(x+h) - F(x)}{h} = \frac{1}{h} \int_x^{x+h} f(t)dt. По теоремата за средната стойност това е равно на f(c) за някое c \in [x, x+h]. При h \to 0, c \to x и поради непрекъснатостта на f, границата е f(x).
2. **Втора част (Формула):** Ако \Phi е коя да е примитивна на f в [a, b], то: \int_a^b f(x)dx = \Phi(b) - \Phi(a)

Приложение за изчисляване

За да се пресметне \int_1^2 3x^2 dx:

1. Намираме примитивна \Phi(x) = x^3.
2. Прилагаме формулата: \Phi(2) - \Phi(1) = 2^3 - 1^3 = 8 - 1 = 7.

8. Глосарий на ключовите термини

- **Разбиване (Partition):** Множество от точки, разделящи интервал на краен брой подинтервали.
- **Сума на Дарбу (Darboux Sum):** Приближение на лицето под графика чрез правоъгълници, базирани на инфимума (малка сума) или супремума (голяма сума) на функцията.
- **Инфимум (Infimum):** Най-голямата долна граница на множество.
- **Супремум (Supremum):** Най-малката горна граница на множество.
- **Равномерна непрекъснатост:** Свойство на функцията, при което малка промяна в аргумента води до малка промяна в стойността, като тази зависимост (делта) не зависи от конкретната точка в интервала.
- **Примитивна функция:** Функция, чиято производна е равна на дадената функция f(x).
- **Теорема на Кантор:** Твърдение, че всяка непрекъсната функция в затворен и краен интервал е равномерно непрекъсната.
- **Теорема на Болцано:** Непрекъсната функция приема всички стойности между своя максимум и минимум в даден интервал.

27

27. Собствени вектори, собствени стойности и характеристични полиноми

1. Характеристичен полином и полиномиална теория

Според източниците, полиномите на една променлива x са фундаментални за дефинирането на характеристичните свойства на операторите.

Определение на характеристичен полином

В контекста на линейните оператори, характеристичният полином се разглежда като полином от вида: f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 където коефициентите a_i принадлежат на определено поле F (обикновено \mathbb{R} или \mathbb{C}).

Ключови свойства от източниците:

- **Степен на полинома:** Означава се с \deg f.
- **Корени:** Елементът \alpha \in K (където K е разширение на полето F) е корен на полинома f(x), ако f(\alpha) = 0 [SOURCE_IMAGE_51].
- **Деление с остатък:** За всеки два полинома f и g \neq 0, съществуват единствени полиноми q (частно) и r (остатък), такива че f = gq + r, където \deg r < \deg g [SOURCE_IMAGE_25].

Схема на Хорнер

Това е алгоритмичен метод за пресмятане на стойността на полином в дадена точка \alpha и за деление на полином на линейния двучлен (x - \alpha). Източниците посочват, че остатъкът при това деление е равен на стойността на полинома в точката f(\alpha) [SOURCE_IMAGE_28].

2. Линейни системи и собствени вектори

Собствените вектори на един линеен оператор са тясно свързани с хомогенните системи линейни уравнения (ХСЛУ).

Хомогенни системи (ХСЛУ)

Една система от вида AX = 0 винаги е съвместима, тъй като притежава поне тривиалното решение \vec{0} [SOURCE_IMAGE_13].

- **Ненулеви решения:** Системата има ненулеви решения тогава и само тогава, когато рангът на матрицата r(A) е по-малък от броя на неизвестните n.
- **Фундаментална система от решения (ФСР):** Всеки базис на пространството от решения на ХСЛУ се нарича фундаментална система от решения. Размерността на това пространство е n - r, където n е броят на неизвестните, а r е рангът на матрицата [SOURCE_IMAGE_14].

Връзка със собствените вектори

Собствените вектори, съответстващи на дадена собствена стойност \lambda, се явяват решения на хомогенна система, чиято матрица е получена чрез характеристичния полином. Линейната независимост на собствените вектори, съответстващи на различни собствени стойности, е ключов теоретичен резултат.

3. Подобни матрици и ранг

Макар източниците да се фокусират върху ранга, те предоставят основата за разбиране на инвариантността при подобни матрици.

- **Ранг на матрица:** Рангът r(A) е редът на най-високия ненулев минор на матрицата [SOURCE_IMAGE_2].
- **Транспониране:** r(A) = r(A^t), тъй като детерминантата не се променя при транспониране [SOURCE_IMAGE_2].
- **Инвариантност:** Подобните матрици имат еднакви характеристични полиноми, което означава, че имат едни и същи собствени стойности (характеристични корени).

4. \phi-инвариантни подпространства

Подпространството W на линейното пространство V се нарича \phi-инвариантно по отношение на линейния оператор \phi, ако за всеки вектор w \in W неговото изображение \phi(w) също принадлежи на W.

Основни теореми за съществуване

В източниците се посочват следните важни теореми без доказателство:

1. **Над полето на комплексните числа (\mathbb{C}):** Във всяко крайномерно линейно пространство над \mathbb{C} съществува поне едно \phi-инвариантно подпространство с размерност 1 (това е подпространството, породено от собствен вектор).
2. **Над полето на реалните числа (\mathbb{R}):** Аналогично твърдение е валидно за пространства с малка размерност (например съществуват инвариантни подпространства с размерност 1 или 2) [SOURCE_TEXT_Tema_27].

Глосарий на ключовите термини

Термин	Дефиниция
Собствен вектор	Ненулев вектор, който при прилагане на линеен оператор променя само мащаба си (умножава се със собствена стойност).
Собствена стойност	Скалар \lambda, за който съществува ненулев вектор v, такъв че \phi(v) = \lambda v.

Характеристичен полином	Полином, чиито корени са собствените стойности на дадена матрица или оператор.
Ранг на матрица	Максималният брой линейно независими редове или стълбове в матрицата.
Хомогенна система	Система от линейни уравнения, в която свободните членове са нули (AX = 0).
ФСР	Фундаментална система от решения; базис на пространството от решения на хомогенна система.
\phi-инвариантно подпространство	Подпространство, което остава затворено под действието на линейния оператор \phi.
Неразложим полином	Полином, който не може да бъде представен като произведение на два полинома от по-ниска степен над дадено поле [SOURCE_IMAGE_44].

28

28. Уравнения на права в равнината

1. Векторно-параметрично уравнение на права

Всяка права g в равнината може да бъде еднозначно определена чрез фиксирана точка M_0, лежаща на нея, и ненулев вектор \vec{p}, който е колинеарен с правата. Векторът \vec{p} се нарича направляващ вектор на правата.

Дефиниция и извеждане: Произволна точка M лежи върху правата g тогава и само тогава, когато векторът \vec{M_0M} е колинеарен с вектора \vec{p}. Това условие се записва като: \vec{M_0M} = \lambda \cdot \vec{p} Където \lambda е реален параметър, който се мени в интервала (-\infty, +\infty).

Като използваме радиус-векторите на точките спрямо началото на координатна система O, можем да запишем: \vec{M_0M} = \vec{OM} - \vec{OM_0} = \lambda \cdot \vec{p} Оттук получаваме **векторно-параметричното уравнение на права**: \vec{r} = \vec{r_0} + \lambda \cdot \vec{p}

2. Координатни (скаларни) параметрични уравнения

Когато фиксираме афинна координатна система K = \{O, \vec{e_1}, \vec{e_2}\}, векторното уравнение може да бъде разложено на компоненти (скалари). Нека точките и векторът имат следните координати:

- M(x, y) — произволна точка от правата;
- M_0(x_0, y_0) — фиксирана точка;
- \vec{p}(a, b) — направляващ вектор.

Тогава правата g се описва от системата **координатни параметрични уравнения**: g: \begin{cases} x = x_0 + \lambda \cdot a \ y = y_0 + \lambda \cdot b \end{cases} Параметърът \lambda определя позицията на всяка точка върху правата. Ако правата се разглежда като ос, то \lambda = \frac{M_0M}{p}.

3. Общо уравнение на права в равнината

Общото уравнение е декартово представяне на правата, получено чрез елиминиране на параметъра \lambda или чрез условието за колинеарност.

Теорема за общо уравнение

Всяка права в равнината спрямо произволна координатна система K = \{O, \vec{e_1}, \vec{e_2}\} има уравнение от вида: Ax + By + C = 0 където (A, B) \neq (0, 0), т.е. A^2 + B^2 \neq 0.

Основни зависимости:

- Векторът \vec{p}(-B, A) е винаги колинеарен с правата.
- Една и съща права може да има безброй много уравнения от този вид. Уравненията Ax + By + C = 0 и A_1x + B_1y + C_1 = 0 определят една и съща права тогава и само тогава,

когато съществува число \lambda \neq 0, такова че: A_1 = \lambda A, \quad B_1 = \lambda B, \quad C_1 = \lambda C

4. Нормално уравнение на права в равнината

Нормалното уравнение се използва най-често в ортонормирани координатни системи. То се базира на нормалния вектор \vec{n}(A, B), който е перпендикулярен на правата g.

Извеждане: Ако имаме общо уравнение Ax + By + C = 0, векторът \vec{n}(A, B) е перпендикулярен на правата. За да получим нормалното уравнение, умножаваме общото уравнение по нормиращ множител \pm \frac{1}{\sqrt{A^2 + B^2}}.

Вид на нормалното уравнение: x \cdot \cos\varphi_0 + y \cdot \sin\varphi_0 + C' = 0 където \cos\varphi_0 = \frac{A}{\sqrt{A^2 + B^2}} и \sin\varphi_0 = \frac{B}{\sqrt{A^2 + B^2}}. Всяка права има две нормални уравнения, различаващи се по знак.

5. Разстояние от точка до права

Разстоянието от точка M_1(x_1, y_1) до права g е дължината на перпендикуляра, спуснат от точката към правата.

Формули за изчисление:

1. Ако правата е зададена с **нормално уравнение** x \cdot \cos\varphi + y \cdot \sin\varphi + c = 0: d = |x_1 \cdot \cos\varphi + y_1 \cdot \sin\varphi + c|
2. Ако правата е зададена с **общо уравнение** Ax + By + C = 0: d = \frac{|Ax_1 + By_1 + C|}{\sqrt{A^2 + B^2}}

Величината \delta = x_1 \cdot \cos\varphi + y_1 \cdot \sin\varphi + c се нарича **ориентирано разстояние**. Знакът на \delta показва в коя полуравнина спрямо правата се намира точката.

6. Полуравнини

Всяка права g: l(x, y) = Ax + By + C = 0 разделя равнината на две отворени полуравнини.

Математическо описание:

- Първа полуравнина \lambda^+: Ax + By + C > 0
- Втора полуравнина \lambda^-: Ax + By + C < 0

Теорема за разположение на две точки: Две точки M_1(x_1, y_1) и M_2(x_2, y_2) лежат в **различни** полуравнини спрямо правата g тогава и само тогава, когато произведението на стойностите на техните леви страни в уравнението е отрицателно: l(M_1) \cdot l(M_2) < 0 Ако l(M_1) \cdot l(M_2) > 0, точките лежат в една и съща полуравнина.

---

7. Взаимни положения между две прави

Нека са дадени две прави: g_1: A_1x + B_1y + C_1 = 0 с направляващ вектор \vec{p_1}(-B_1, A_1) g_2: A_2x + B_2y + C_2 = 0 с направляващ вектор \vec{p_2}(-B_2, A_2)

Положение	Условие (чрез коефициенти)	Условие (чрез вектори)
Успоредни (g_1 \parallel g_2)	A_1 = \lambda A_2, B_1 = \lambda B_2, C_1 \neq \lambda C_2	\vec{p_1} и \vec{p_2} са колинеарни
Съвпадащи (g_1 \equiv g_2)	A_1 = \lambda A_2, B_1 = \lambda B_2, C_1 = \lambda C_2	\vec{p_1} \parallel \vec{p_2} и обща точка
Пресичащи се	\frac{A_1}{A_2} \neq \frac{B_1}{B_2}	\vec{p_1} и \vec{p_2} не са колинеарни

Сноп прави

Множеството от всички прави, минаващи през пресечната точка M на две прави g_1 и g_2, се нарича **сноп прави**. Уравнението на всяка права от този сноп има вида: \alpha \cdot l_1(x, y) + \beta \cdot l_2(x, y) = 0 където \alpha и \beta са реални параметри, не едновременно нули.

Речник на основните термини

- **Направляващ вектор (\vec{p}):** Ненулев вектор, чиято посока съвпада с посоката на правата или е успоредна на нея.
- **Параметрично уравнение:** Представяне на координатите на точките от правата като функции на един независим параметър (\lambda).
- **Общо уравнение:** Уравнение от вида Ax + By + C = 0, което дефинира всички точки от правата в декартови координати.
- **Нормален вектор (\vec{n}):** Вектор, който е перпендикулярен на всяка отсечка от правата. В общото уравнение това е векторът с компоненти (A, B).
- **Ориентирано разстояние (\delta):** Стойност, чийто модул е разстоянието до правата, а знакът показва в коя страна на равнината спрямо правата се намира точката.
- **Сноп прави:** Колекция от всички прави в равнината, които се пресичат в една обща точка (център на снопа).
- **Колинеарни вектори:** Вектори, които лежат на една и съща права или на успоредни прави.
- **Просто отношение (\lambda = (ABM)):** Число, дефиниращо разположението на точка M спрямо точки A и B върху една права; \lambda < 0 означава, че M е между A и B.

29. Аналитична геометрия: Уравнения на права и равнина в пространството

1. Уравнения на права в пространството

Една права g в пространството се определя от фиксирана точка M_0 върху нея и ненулев вектор p, който е успореден (колинеарен) на правата.

Векторно-параметрично уравнение

За произволна точка M, лежаща на правата, векторът \vec{M_0M} трябва да бъде колинеарен с вектора p.

- **Условие:** \vec{M_0M} = \lambda \cdot p
- **Уравнение:** r = r_0 + \lambda \cdot p
    - Където r е радиус-векторът на текущата точка M, r_0 е радиус-векторът на фиксираната точка M_0, а \lambda е параметър, който се мени в интервала (-\infty, +\infty).

Координатни (скаларни) параметрични уравнения

Ако фиксираме координатна система K\{O, e_1, e_2, e_3\} и имаме координати M(x, y, z), M_0(x_0, y_0, z_0) и p(a, b, c), правата се описва със системата: g: \begin{cases} x = x_0 + \lambda a \ y = y_0 + \lambda b \ z = z_0 + \lambda c \end{cases}

---

2. Уравнения на равнина в пространството

Една равнина \alpha се дефинира чрез фиксирана точка M_0 \in \alpha и два ненулеви, неколинеарни вектора p и q, които са компланарни (успоредни) на равнината.

Векторно-параметрично уравнение на равнина

Всяка точка M лежи в равнината тогава и само тогава, когато векторът \vec{M_0M} може да се представи като линейна комбинация на p и q:

- **Уравнение:** r = r_0 + \lambda \cdot p + \mu \cdot q
    - Тук \lambda и \mu са реални параметри.

Координатни (скаларни) параметрични уравнения на равнина

Спрямо координатна система K, ако M_0(x_0, y_0, z_0), p(p_1, p_2, p_3) и q(q_1, q_2, q_3), уравненията са: \alpha: \begin{cases} x = x_0 + \lambda p_1 + \mu q_1 \ y = y_0 + \lambda p_2 + \mu q_2 \ z = z_0 + \lambda p_3 + \mu q_3 \end{cases}

---

3. Теорема за общо уравнение на равнина

Теорема: Всяка равнина спрямо фиксирана афинна координатна система има общо уравнение от вида: Ax + By + Cz + D = 0 където (A, B, C) \neq (0, 0, 0).

Извеждане

Общото уравнение се извежда от условието за компланарност на векторите \vec{M_0M}, p и q. Това означава, че детерминантата, съставена от техните координати, трябва да е равна на нула: \begin{vmatrix} x - x_0 & y - y_0 & z - z_0 \ p_1 & p_2 & p_3 \ q_1 & q_2 & q_3 \end{vmatrix} = 0 Развивайки детерминантата, се получават коефициентите:

- A = p_2q_3 - p_3q_2
- B = p_3q_1 - p_1q_3
- C = p_1q_2 - p_2q_1
- D = -Ax_0 - By_0 - Cz_0

Важно: Две уравнения задават една и съща равнина, ако техните коефициенти са пропорционални: A_1 = \lambda A_2, B_1 = \lambda B_2, C_1 = \lambda C_2, D_1 = \lambda D_2.

---

4. Взаимни положения между две равнини

Нека са дадени две равнини: \alpha_1: A_1x + B_1y + C_1z + D_1 = 0 \alpha_2: A_2x + B_2y + C_2z + D_2 = 0

Взаимно положение	Условие
Успоредни (\alpha_1 \parallel \alpha_2)	A_1 = kA_2, B_1 = kB_2, C_1 = kC_2, но D_1 \neq kD_2
Съвпадащи (\alpha_1 \equiv \alpha_2)	A_1 = kA_2, B_1 = kB_2, C_1 = kC_2, D_1 = kD_2
Пресичащи се	Коефициентите пред пред x, y, z не са пропорционални.

---

5. Нормално уравнение на равнина и разстояние от точка до равнина

Нормално уравнение

Ако координатната система е ортонормирана, векторът N(A, B, C) е перпендикулярен на равнината (нормален вектор). Нормалното уравнение се получава чрез делене на общото уравнение на дължината на нормалния вектор: \frac{Ax + By + Cz + D}{\pm \sqrt{A^2 + B^2 + C^2}} = 0 Векторът N_0 = \frac{N}{|N|} е единичен нормален вектор.

Разстояние от точка до равнина

Разстоянието от точка M_1(x_1, y_1, z_1) до равнина \alpha: Ax + By + Cz + D = 0 се изчислява по формулата: |M_1, \alpha| = \frac{|Ax_1 + By_1 + Cz_1 + D|}{\sqrt{A^2 + B^2 + C^2}}

- **Ориентирано разстояние (\delta):** Стойността без модул. Знакът на \delta показва разположението на точката спрямо нормалния вектор и равнината.

---

6. Полупространства

Всяка равнина \alpha: l(x, y, z) = Ax + By + Cz + D = 0 разделя пространството на две отворени полупространства:

1. \mu_1: Ax + By + Cz + D > 0
2. \mu_2: Ax + By + Cz + D < 0

Критерий за разположение: Две точки M_1 и M_2 лежат в различни полупространства относно \alpha тогава и само тогава, когато произведенията на стойностите на техните уравнения имат отрицателен знак: l(M_1) \cdot l(M_2) < 0

---

7. Представяне на права чрез две равнини

Една права g може да бъде дефинирана като сечение на две непаралелни равнини: g: \begin{cases} A_1x + B_1y + C_1z + D_1 = 0 \ A_2x + B_2y + C_2z + D_2 = 0 \end{cases} Условието за коректност е векторите (A_1, B_1, C_1) и (A_2, B_2, C_2) да не са пропорционални.

Канонично представяне

Правата може да се представи и в каноничен вид чрез двойка равнини (ако равнините не са успоредни): g: \{ y = ax + b; z = cx + d \}

Речник на ключовите термини

- **Колинеарни вектори:** Вектори, които лежат на една и съща права или на успоредни прави.
- **Компланарни вектори:** Вектори, които са успоредни на една и съща равнина.
- **Нормален вектор:** Вектор, който е перпендикулярен на всяка права в дадена равнина.
- **Ориентирано разстояние:** Число, чийто модул е разстоянието от точка до равнина, а знакът показва разположението на точката спрямо нормалния вектор.

- **Параметрично уравнение:** Уравнение, в което координатите на точките са изразени чрез помощни променливи (параметри).
- **Трансверзала:** Права, която пресича две или повече други прави (често използвана при задачи с кръстосани прави).
- **Ортонормирана координатна система:** Координатна система, в която координатните оси са взаимно перпендикулярни и единичните вектори по тях имат дължина 1.

30

30. Дискретни вероятностни разпределения

1. Основи на теорията на вероятностите и случайните величини

Дефиниция на вероятност и свойства

Вероятността (\mathbb{P}) е нормирано изображение от \sigma-алгебра \mathcal{A} върху множество от елементарни събития \Omega към интервала [0, 1]. За да бъде едно изображение вероятност, то трябва да изпълнява следните аксиоми:

1. **Неотрицателност:** \mathbb{P}(A) \ge 0 за всяко събитие A \in \mathcal{A}.
2. **Нормираност:** \mathbb{P}(\Omega) = 1.
3. **\sigma-адитивност:** За редица от непресичащи се (дизюнктни) събития A_1, A_2, \dots, вероятността на тяхното обединение е сумата от техните вероятности: \mathbb{P}(\bigcup_{k=1}^{\infty} A_k) = \sum_{k=1}^{\infty} \mathbb{P}(A_k).

Основни свойства:

- **Вероятност на невъзможното събитие:** \mathbb{P}(\emptyset) = 0.
- **Допълнение:** \mathbb{P}(A^c) = 1 - \mathbb{P}(A).
- **Монотонност:** Ако A \subseteq B, то \mathbb{P}(A) \le \mathbb{P}(B).
- **Адитивност за две събития:** \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B).

Случайна величина (СВ)

Случайната величина X е функция, която съпоставя реално число на всеки елементарен изход \omega \in \Omega.

- **Дискретна случайна величина (ДСВ):** Приема краен или изброимо голям брой стойности.
- **Разпределение на ДСВ:** Представя се чрез таблица, съпоставяща стойностите x_i на съответните им вероятности p_i = \mathbb{P}(X = x_i), като \sum p_i = 1.

2. Числови характеристики и пораждащи функции

Математическото очакване и Дисперсия

- **Математическо очакване (\mathbb{E}X):** Претеглена средна стойност, дефинирана като \mathbb{E}X = \sum x_i p_i. Тя представлява „центъра“ на разпределението.
- **Дисперсия (\mathbb{D}X):** Измерва разсейването около очакването. Дефинира се като \mathbb{D}X = \mathbb{E}(X - \mathbb{E}X)^2 = \mathbb{E}X^2 - (\mathbb{E}X)^2.

Пораждащи функции (Инструменти за изчисление)

1. **Пораждаща функция на вероятностите (g_X(s)):** Дефинирана за целочислени, неотрицателни ДСВ като g_X(s) = \mathbb{E}s^X = \sum_{k=0}^{\infty} s^k \mathbb{P}(X=k).
    - Свойство 1: g_X(1) = 1.

    - Свойство 2: \mathbb{E}X = g'_X(1).
    - Свойство 3: \mathbb{D}X = g''_X(1) + g'_X(1) - (g'_X(1))^2.
2. **Функция на моментите (M_X(t)):** M_X(t) = \mathbb{E}e^{tX}. Приложима и за непрекъснати величини. Производните ѝ в точката t=0 дават моментите на СВ: M_X^{(k)}(0) = \mathbb{E}X^k.

3. Анализ на основните дискретни разпределения

В таблицата по-долу са синтезирани ключовите данни за четирите разпределения, изисквани за изпита:

Разпределение	Дефиниция / Вероятност \mathbb{P}(X=k)	Пример за възникване	\mathbb{E}X	\mathbb{D}X
Равномерно	\frac{1}{n} за всяко k в крайно множество	Хвърляне на честен зар; падане на снежинка в участък.	\frac{a+b}{2}	\frac{n^2-1}{12}
Биномно	\binom{n}{k} p^k (1-p)^{n-k}	Брой успехи в n независими опита (напр. брой „ези“).	np	np(1-p)
Геометрично	(1-p)^k p (брой неуспехи до успех)	Брой пропуснати удари на бейзболист до първия хит.	\frac{1-p}{p}	\frac{1-p}{p^2}
Поасоново	e^{-\lambda} \frac{\lambda^k}{k!}	Брой заявки към сървър или катастрофи за време t.	\lambda	\lambda

Детайли по разпределенията:

- **Биномно:** Моделира броя успехи. Ако X = \sum X_i (където X_i са бернулиеви), то \mathbb{E}X = \sum \mathbb{E}X_i = np.
- **Геометрично:** Притежава свойството **„безпаметност“** – вероятността за бъдещ успех не зависи от броя на миналите неуспехи.
- **Поасоново:** Използва се за „редки събития“. Чрез него може да се апроксимира биномното разпределение при голямо n и малко p (\lambda = np).
- **Равномерно (непрекъснато):** Плътността е f(x) = \frac{1}{b-a} за x \in (a, b). Очакването е средата на интервала.

Глосарий на ключовите термини

- **\sigma-алгебра:** Колекция от подмножества на \Omega, затворена спрямо допълнение и изброимо обединение.
- **Адитивност:** Свойство, при което вероятността на обединението на несъвместими събития е равна на сумата от техните индивидуални вероятности.
- **Бернулиев опит:** Случаен експеримент с точно два възможни изхода – „успех“ (с вероятност p) и „неуспех“ (с вероятност 1-p).
- **Дисперсия:** Статистическа мярка за разсейването на стойностите на случайна величина около нейната средна стойност.
- **Математическо очакване:** Теоретичната средна стойност на случайна величина при безкраен брой повторения на експеримента.
- **Монотонност на вероятността:** Ако събитие A е част от събитие B, то вероятността на A не надвишава тази на B.
- **Нормировка:** Условието сумата от вероятностите на всички възможни изходи да бъде точно 1.
- **Пораждаща функция:** Математически инструмент (степенен ред), който кодира вероятностите на ДСВ и улеснява изчисляването на моменти.
- **Случайна величина:** Функция, дефинирана върху вероятностно пространство, която приема числови стойности в зависимост от изхода на случаен процес.